MINISTERUL EDUCAȚIEI ȘI CERCETĂRII AL REPUBLICII MOLDOVA

UNIVERSITATEA LIBERĂ INTERNAȚIONALĂ DIN MOLDOVA

ULIM



Facultatea Informatică, Inginerie și Design

Catedra Tehnologii Informaționale

RAPORT

Lucrarea de laborator nr. 1 · Varianta 1

Disciplina: Bazele Programării în Java

Tema: Instrucțiuni liniare

Anul de studii al cerințelor: 2024–2025

A elaborat: Alexa Adrian · Grupa TI-262-21

Profesor: Mititelu Vitalii

Data prezentării: _____

Nota: _____ Semnătura profesorului: _____

Chișinău

Scopul și sarcinile lucrării

Citirea unei valori reale x și calcularea expresiei y = (sin x + 2x) / (x³ − 2x). Demonstrarea matematică a rezultatului și analiza domeniului de definiție.

Material teoretic

Un algoritm liniar execută operațiile într-o ordine fixă: citește datele, evaluează expresia și afișează rezultatul. În Java, atribuirea memorează rezultatul unei expresii într-o variabilă declarată. Tipul double permite reprezentarea aproximativă a numerelor reale în virgulă mobilă. Din acest motiv, rezultatele transcendente sunt aproximări numerice, nu valori simbolice exacte.

Funcția Math.sin(x) primește un unghi în radiani. Expresia 2 * x reprezintă produsul, iar ridicarea la putere se poate scrie prin înmulțiri sau prin Math.pow. Semnul ^ nu este operator de ridicare la putere în Java. Parantezele sunt obligatorii pentru a păstra structura fracției din enunț.

Numitorul se descompune astfel: x³ − 2x = x(x² − 2) = x(x − √2)(x + √2). Domeniul matematic este mulțimea numerelor reale fără valorile 0, √2 și −√2. Sinusul este definit pentru orice număr real, deci nu impune restricții suplimentare. În program, valorile double corespunzătoare rădăcinilor sunt respinse explicit. Pentru vecinătăți foarte apropiate de rădăcini pot apărea valori mari și pierderi de precizie.

Validările protejează calculul, fără a schimba formula cerută. Double.isFinite respinge NaN și infinitul. Nu folosim un prag epsilon arbitrar pentru a declara nule toate numerele mici: un asemenea prag ar elimina și valori matematic valide. Dacă produsul intermediar sau rezultatul nu poate fi reprezentat finit, programul raportează limita numerică.

Convenții de lucru: surse în UTF-8, Java 21, intrare de la tastatură și mesaje în limba română. Separatorul zecimal la tastare este punctul. Cazurile de test au fost executate efectiv; jurnalele brute și codul sunt incluse în varianta electronică.

Algoritmul și verificarea rezultatului

1. Se citește x și se verifică formatul numeric și caracterul finit. 2. Se resping zerourile numitorului. 3. Se calculează numitorul factorizat și expresia. 4. Se verifică limitele double și se afișează 12 zecimale.

Demonstrație pentru x = 1: sin(1) ≈ 0,8414709848078965, numărătorul este 2,8414709848078965, iar numitorul este 1 − 2 = −1. Rezultă y ≈ −2,8414709848078965. Programul afișează −2,841470984808.

Pentru x = −1, atât numărătorul, cât și numitorul își schimbă semnul. Prin urmare y(−1) = y(1), confirmat de a doua rulare. Pentru x = 0, numitorul este nul și nu există o valoare definită. Complexitate: timp O(1), memorie suplimentară O(1).

Planul de testare și rezultatele

Cazul 1: 1. Verificare: „y = -2.841470984808”. Rezultat: trecut; cod de ieșire 0.

Cazul 2: -1. Verificare: „y = -2.841470984808”. Rezultat: trecut; cod de ieșire 0.

Cazul 3: 0. Verificare: „numitor nul”. Rezultat: trecut; cod de ieșire 0.

Cazul 4: 1.4142135623730951. Verificare: „numitor nul”. Rezultat: trecut; cod de ieșire 0.

Cazul 5: abc. Verificare: „Eroare”. Rezultat: trecut; cod de ieșire 0.

Cazul 6: 1e200. Verificare: „limitele”. Rezultat: trecut; cod de ieșire 0.

Pentru fiecare caz sunt păstrate separat intrarea, ieșirea și rezultatul așteptat. Capturile reproduc jurnalele reale într-o pagină de vizualizare; nu simulează un IDE. La lucrarea 9, capturile arată finalul jurnalului, iar fișierele TXT păstrează sesiunea integrală.

Codul integral — Lab01.java

```
import java.util.Locale;
import java.util.Scanner;

public class Lab01 {
    public static void main(String[] args) {
        Locale.setDefault(Locale.ROOT);
        Scanner in = new Scanner(System.in);
        System.out.println("Alexa Adrian | TI-262-21 | Laboratorul 1, varianta 1");
        System.out.println("Introduceți x (în radiani):");
        if (!in.hasNextDouble()) {
            System.out.println("Eroare: introduceți un număr real valid.");
            return;
        }
        double x = in.nextDouble();
        if (!Double.isFinite(x)) {
            System.out.println("Eroare: x trebuie să fie finit.");
            return;
        }
        // x^3 - 2x = x(x - sqrt(2))(x + sqrt(2)).
        double rad = Math.sqrt(2.0);
        if (x == 0.0 || x == rad || x == -rad) {
            System.out.println("Eroare: numitor nul; expresia nu este definită.");
            return;
        }
        double numitor = x * (x - rad) * (x + rad);
        double y = (Math.sin(x) + 2.0 * x) / numitor;
        if (!Double.isFinite(numitor) || !Double.isFinite(y)
                || numitor == 0.0) {
            System.out.println("Eroare: limitele reprezentării double au fost depășite.");
            return;
        }
        System.out.printf("y = %.12f%n", y);
    }
}
```

Capturi de ecran și interpretare

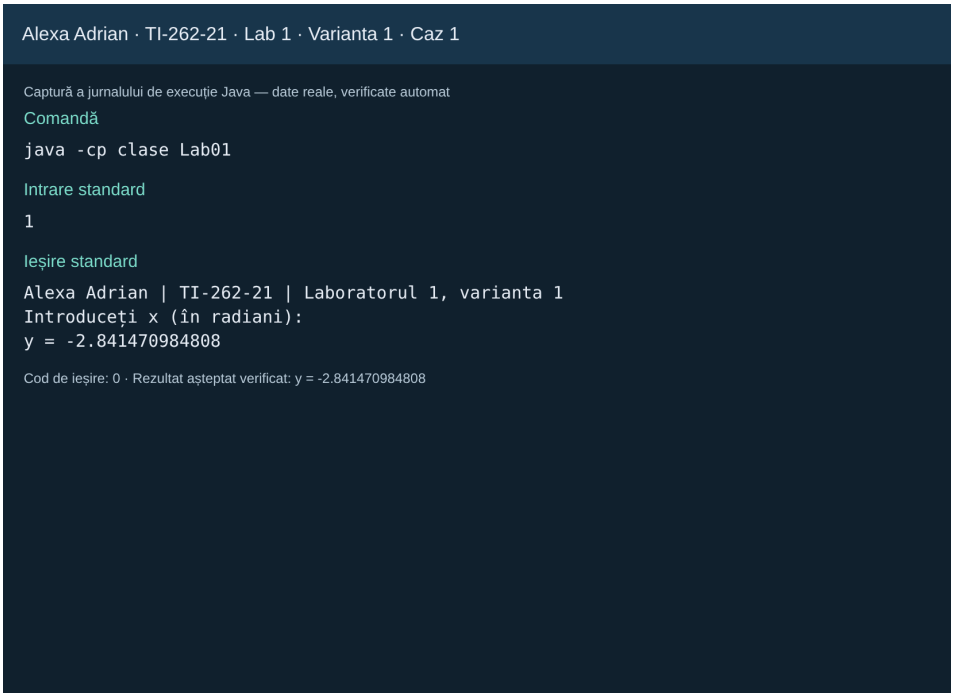


Figura 1. Cazul 1: rezultatul așteptat „y = -2.841470984808” a fost confirmat de execuție.

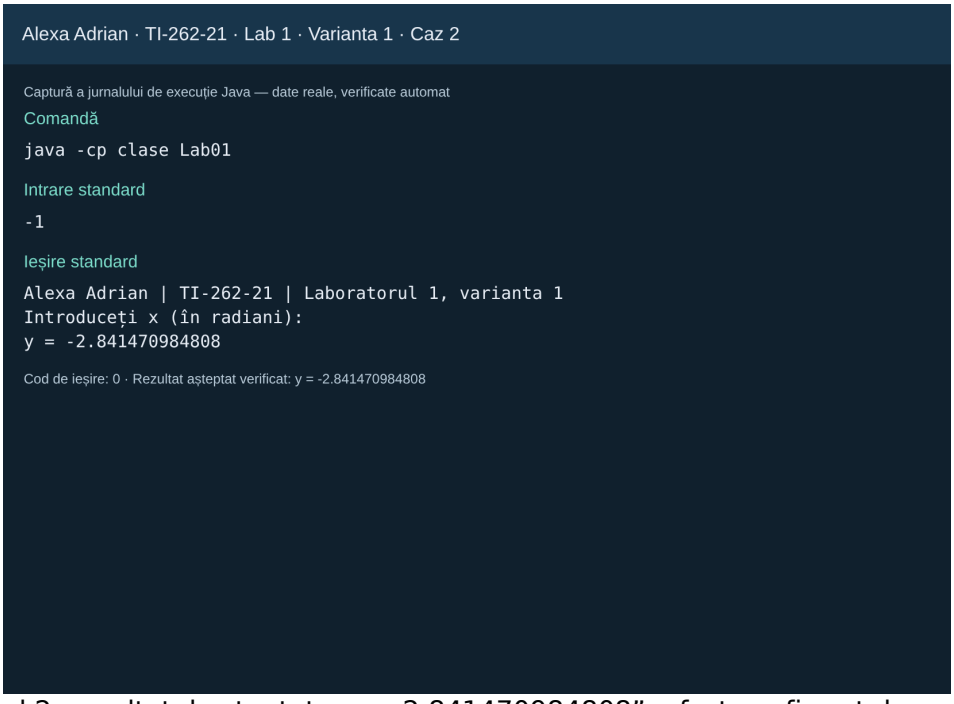


Figura 2. Cazul 2: rezultatul așteptat „y = -2.841470984808” a fost confirmat de execuție.

Capturi de ecran și interpretare



Figura 3. Cazul 3: rezultatul așteptat „numitor nul” a fost confirmat de execuție.

Concluzii

1. Factorizarea numitorului permite determinarea exactă a domeniului matematic.

2. Rulările pentru x = 1 și x = −1 confirmă simetria și rezultatul calculat manual.

3. Verificarea formatului și a valorilor nefinite evită afișarea unui rezultat numeric înșelător.

Întrebări pentru pregătirea susținerii

• De ce Math.sin folosește radiani?

• De ce nu scriem x ^ 3?

• Ce se întâmplă lângă √2?

Reproducerea lucrării

Din rădăcina arhivei: javac -encoding UTF-8 -d clase cod/Lab01.java, apoi java -cp clase Lab01. Este necesar un JDK 21 sau mai nou. Pentru Lab09, un argument opțional stabilește calea fișierului TSV; fără argument se folosește studenti.tsv în directorul curent.

Surse

1. „Lucrările de laborator la disciplina Bazele Programării în Java”, anul 2024–2025, actualizare 16.09.2024, PDF furnizat pentru această lucrare.

2. Documentația Java SE 21: https://docs.oracle.com/en/java/javase/21/docs/api/ — clasele Scanner, Math, String, Files și BigInteger, după caz.

Varianta electronică include sursa integrală, capturile PNG, jurnalele TXT, fișierele de date și instrucțiunile de rulare. Datele de prezentare, nota și semnătura se completează la susținere. Emblema de pe copertă este o reproducere grafică după imaginea furnizată de student.

MINISTERUL EDUCAȚIEI ȘI CERCETĂRII AL REPUBLICII MOLDOVA

UNIVERSITATEA LIBERĂ INTERNAȚIONALĂ DIN MOLDOVA

ULIM



Facultatea Informatică, Inginerie și Design

Catedra Tehnologii Informaționale

RAPORT

Lucrarea de laborator nr. 2 · Varianta 1

Disciplina: Bazele Programării în Java

Tema: Instrucțiuni decizionale (de ramificare)

Anul de studii al cerințelor: 2024–2025

A elaborat: Alexa Adrian · Grupa TI-262-21

Profesor: Mititelu Vitalii

Data prezentării: _____

Nota: _____ Semnătura profesorului: _____

Chișinău

Scopul și sarcinile lucrării

Determinarea maximului a trei numere întregi, folosind instrucțiuni if.

Material teoretic

Instrucțiunea if execută un bloc numai dacă expresia logică este adevărată. Operatorii relaționali, precum > și >=, compară valorile și produc rezultate de tip boolean. O ramificare permite alegerea unui rezultat în funcție de datele de intrare. În această lucrare se folosește if, conform recomandării pentru varianta 1.

Maximul unei mulțimi finite este un element care nu este mai mic decât niciun alt element. Algoritmul pornește de la primul număr și îl tratează ca maxim provizoriu. Dacă al doilea număr este mai mare, maximul se actualizează. Același pas se repetă pentru al treilea număr. Sunt necesare doar două comparații pentru calculul propriu-zis.

Inițializarea cu primul element este esențială. Inițializarea cu zero ar produce un rezultat greșit când toate valorile sunt negative. Egalitatea nu necesită actualizare: dacă două numere sunt egale și maxime, valoarea maximă rămâne aceeași. Cerința solicită valoarea, nu poziția unei apariții.

Tipul int are un interval limitat, de la −2147483648 la 2147483647. Scanner.hasNextInt verifică atât forma, cât și încadrarea valorii. Programul nu adună numerele, deci nu apare depășire aritmetică în calculul maximului. Vectorul de trei elemente este utilizat doar pentru citire; decizia este exprimată explicit prin două instrucțiuni if.

Convenții de lucru: surse în UTF-8, Java 21, intrare de la tastatură și mesaje în limba română. Separatorul zecimal la tastare este punctul. Cazurile de test au fost executate efectiv; jurnalele brute și codul sunt incluse în varianta electronică.

Algoritmul și verificarea rezultatului

După prima atribuire, maxim este egal cu primul număr. După prima comparație, el este maximul primelor două. După a doua comparație, el este maximul tuturor celor trei; această proprietate demonstrează corectitudinea algoritmului.

Pentru 3, 7, 5, maximul provizoriu devine 7 și rămâne 7. Pentru −9, −2, −5, rezultatul corect este −2. Pentru 4, 4, 4, nicio actualizare nu este necesară. Un token nenumeric produce un mesaj de eroare.

Complexitate: O(1) timp și O(1) memorie, deoarece numărul valorilor este fix.

Planul de testare și rezultatele

Cazul 1: 3 7 5. Verificare: „Maximul = 7”. Rezultat: trecut; cod de ieșire 0.

Cazul 2: -9 -2 -5. Verificare: „Maximul = -2”. Rezultat: trecut; cod de ieșire 0.

Cazul 3: 4 4 4. Verificare: „Maximul = 4”. Rezultat: trecut; cod de ieșire 0.

Cazul 4: a. Verificare: „Eroare”. Rezultat: trecut; cod de ieșire 0.

Pentru fiecare caz sunt păstrate separat intrarea, ieșirea și rezultatul așteptat. Capturile reproduc jurnalele reale într-o pagină de vizualizare; nu simulează un IDE. La lucrarea 9, capturile arată finalul jurnalului, iar fișierele TXT păstrează sesiunea integrală.

Codul integral — Lab02.java

```
import java.util.Scanner;

public class Lab02 {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.println("Alexa Adrian | TI-262-21 | Laboratorul 2, varianta 1");
        System.out.println("Introduceți trei numere întregi (int):");
        int[] v = new int[3];
        for (int i = 0; i < 3; i++) {
            if (!in.hasNextInt()) {
                System.out.println("Eroare: sunt necesare trei valori de tip int.");
                return;
            }
            v[i] = in.nextInt();
        }
        int maxim = v[0];
        if (v[1] > maxim) {
            maxim = v[1];
        }
        if (v[2] > maxim) {
            maxim = v[2];
        }
        System.out.println("Maximul = " + maxim);
    }
}
```

Capturi de ecran și interpretare

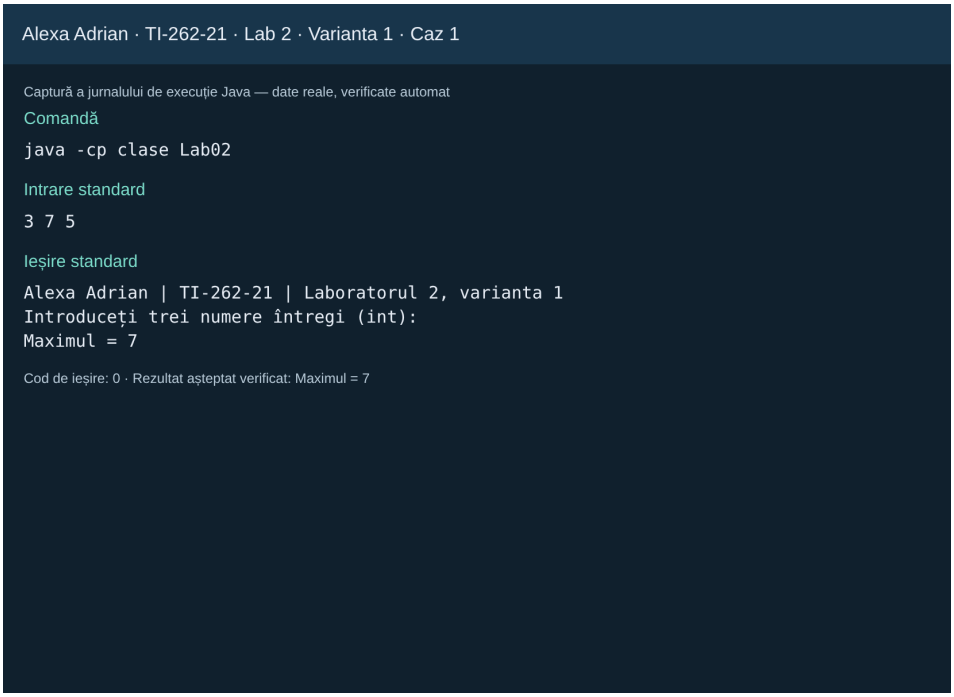


Figura 1. Cazul 1: rezultatul așteptat „Maximul = 7” a fost confirmat de execuție.

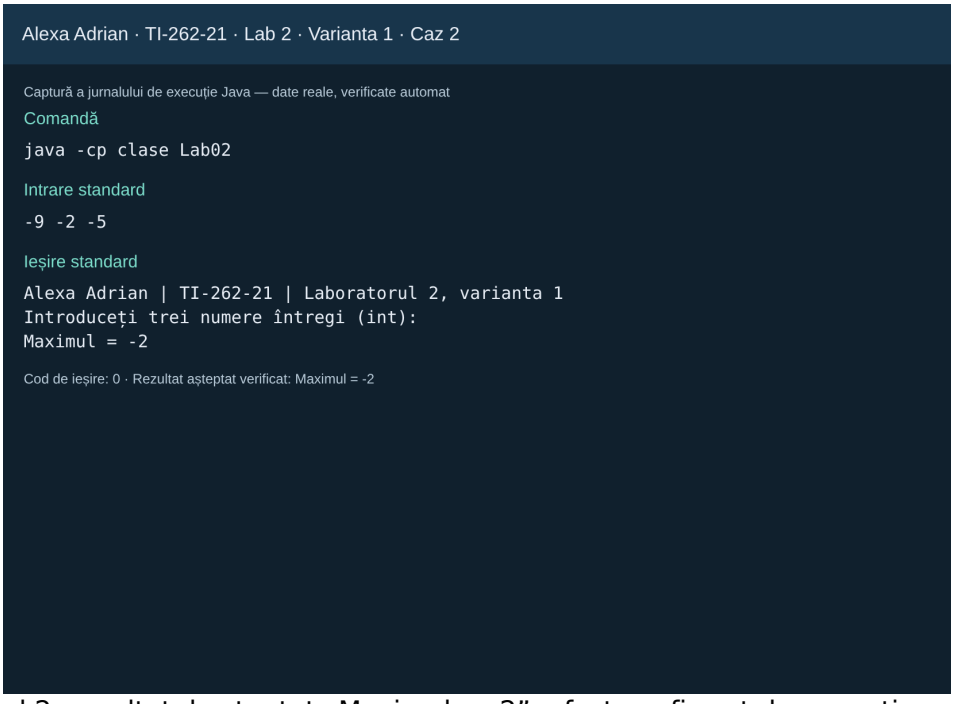


Figura 2. Cazul 2: rezultatul așteptat „Maximul = -2” a fost confirmat de execuție.

Capturi de ecran și interpretare

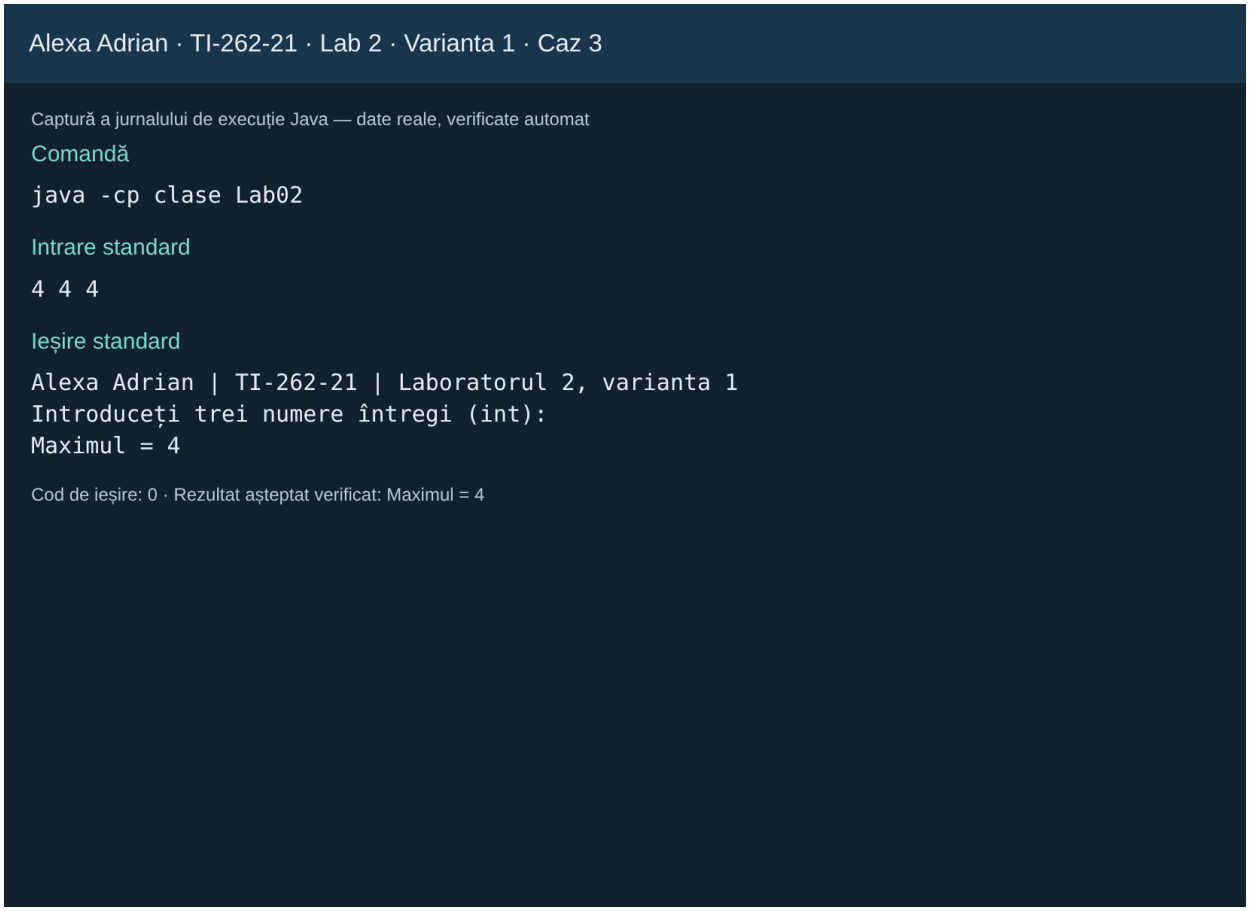


Figura 3. Cazul 3: rezultatul așteptat „Maximul = 4” a fost confirmat de execuție.

Concluzii

1. Inițializarea maximului cu primul număr asigură corectitudinea și pentru date negative.

2. Două comparații sunt suficiente pentru trei valori.

3. Egalitățile și intrările invalide trebuie tratate explicit în verificarea programului.

Întrebări pentru pregătirea susținerii

• De ce nu inițializăm maximul cu zero?

• Ce se întâmplă pentru trei valori egale?

• Care este intervalul tipului int?

Reproducerea lucrării

Din rădăcina arhivei: javac -encoding UTF-8 -d clase cod/Lab02.java, apoi java -cp clase Lab02. Este necesar un JDK 21 sau mai nou. Pentru Lab09, un argument opțional stabilește calea fișierului TSV; fără argument se folosește studenti.tsv în directorul curent.

Surse

1. „Lucrările de laborator la disciplina Bazele Programării în Java”, anul 2024–2025, actualizare 16.09.2024, PDF furnizat pentru această lucrare.

2. Documentația Java SE 21: https://docs.oracle.com/en/java/javase/21/docs/api/ — clasele Scanner, Math, String, Files și BigInteger, după caz.

Varianta electronică include sursa integrală, capturile PNG, jurnalele TXT, fișierele de date și instrucțiunile de rulare. Datele de prezentare, nota și semnătura se completează la susținere. Emblema de pe copertă este o reproducere grafică după imaginea furnizată de student.

MINISTERUL EDUCAȚIEI ȘI CERCETĂRII AL REPUBLICII MOLDOVA

UNIVERSITATEA LIBERĂ INTERNAȚIONALĂ DIN MOLDOVA

ULIM



Facultatea Informatică, Inginerie și Design

Catedra Tehnologii Informaționale

RAPORT

Lucrarea de laborator nr. 3 · Varianta 1

Disciplina: Bazele Programării în Java

Tema: Instrucțiuni repetitive (cicluri)

Anul de studii al cerințelor: 2024–2025

A elaborat: Alexa Adrian · Grupa TI-262-21

Profesor: Mititelu Vitalii

Data prezentării: _____

Nota: _____ Semnătura profesorului: _____

Chișinău

Scopul și sarcinile lucrării

Calcularea sumei numerelor pare de la 2 la n, utilizând instrucțiunea for.

Material teoretic

Instrucțiunea for grupează inițializarea, condiția de continuare și actualizarea contorului. Corpul ciclului se execută cât timp condiția este adevărată. Pentru parcurgerea numerelor pare, contorul pornește de la 2 și crește cu 2. Astfel, nu mai este necesar un test de paritate în interiorul ciclului.

Variabila acumulatoare conține suma parțială. Valoarea ei inițială este zero, elementul neutru al adunării. După fiecare iterație, acumulatorul este suma tuturor numerelor pare deja parcurse. Dacă n este 0 sau 1, corpul nu se execută și suma mulțimii vide este zero.

Numerele naturale impare nu se adaugă, dar n poate fi impar: pentru n = 9 se adună 2 + 4 + 6 + 8. Programul respinge valorile negative, deoarece interfața definește n ca întreg nenegativ. Această convenție elimină ambiguitatea unui interval cu capăt superior negativ.

BigInteger reprezintă întregi cu precizie arbitrară. Metoda add întoarce un obiect nou, deci rezultatul trebuie atribuit din nou variabilei suma. Utilizarea sa evită depășirea tipului int la însumare. Limita de un milion pentru n este o limită practică de timp, declarată în mesajul de intrare. Algoritmul rămâne unul iterativ, conform cerinței, chiar dacă există și o formulă directă de verificare.

Convenții de lucru: surse în UTF-8, Java 21, intrare de la tastatură și mesaje în limba română. Separatorul zecimal la tastare este punctul. Cazurile de test au fost executate efectiv; jurnalele brute și codul sunt incluse în varianta electronică.

Algoritmul și verificarea rezultatului

Algoritm: se citește n, se validează intervalul, se inițializează suma cu zero și se adună 2, 4, 6, … până la n inclusiv.

Dacă k = ⌊n / 2⌋, atunci S = 2(1 + 2 + … + k) = k(k + 1). Formula este utilizată ca verificare independentă, nu în locul ciclului.

Pentru n = 10, k = 5 și S = 5 · 6 = 30. Pentru n = 1 rezultă zero. Pentru n = 1000000, S = 500000 · 500001 = 250000500000, o valoare care depășește int.

Ciclul efectuează ⌊n/2⌋ iterații. În limitele declarate, costul este O(n) operații de acumulare; BigInteger ocupă spațiu proporțional cu numărul de biți ai sumei.

Planul de testare și rezultatele

Cazul 1: 10. Verificare: „= 30”. Rezultat: trecut; cod de ieșire 0.

Cazul 2: 1. Verificare: „= 0”. Rezultat: trecut; cod de ieșire 0.

Cazul 3: -4. Verificare: „Eroare”. Rezultat: trecut; cod de ieșire 0.

Cazul 4: 1000000. Verificare: „= 250000500000”. Rezultat: trecut; cod de ieșire 0.

Pentru fiecare caz sunt păstrate separat intrarea, ieșirea și rezultatul așteptat. Capturile reproduc jurnalele reale într-o pagină de vizualizare; nu simulează un IDE. La lucrarea 9, capturile arată finalul jurnalului, iar fișierele TXT păstrează sesiunea integrală.

Codul integral — Lab03.java

```
import java.math.BigInteger;
import java.util.Scanner;

public class Lab03 {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.println("Alexa Adrian | TI-262-21 | Laboratorul 3, varianta 1");
        System.out.println("Introduceți n întreg nenegativ (maximum 1000000):");
        if (!in.hasNextInt()) {
            System.out.println("Eroare: n trebuie să fie întreg.");
            return;
        }
        int n = in.nextInt();
        if (n < 0 || n > 1000000) {
            System.out.println("Eroare: n trebuie să aparțină intervalului [0, 1000000].");
            return;
        }
        BigInteger suma = BigInteger.ZERO;
        for (int i = 2; i <= n; i += 2) {
            suma = suma.add(BigInteger.valueOf(i));
        }
        System.out.println("Suma numerelor pare = " + suma);
    }
}
```

Capturi de ecran și interpretare

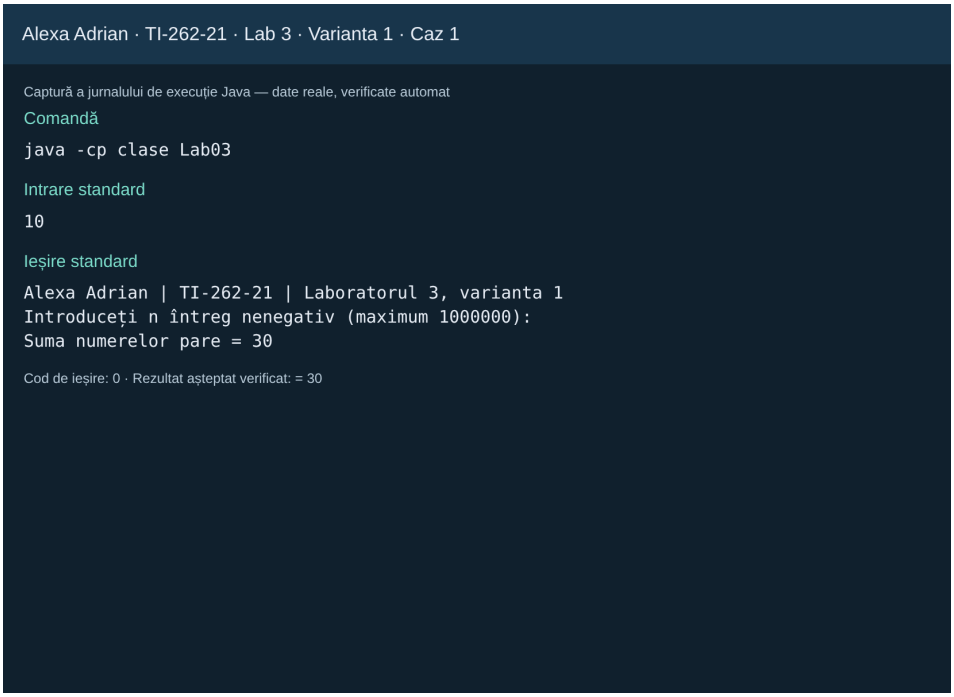


Figura 1. Cazul 1: rezultatul așteptat „= 30” a fost confirmat de execuție.

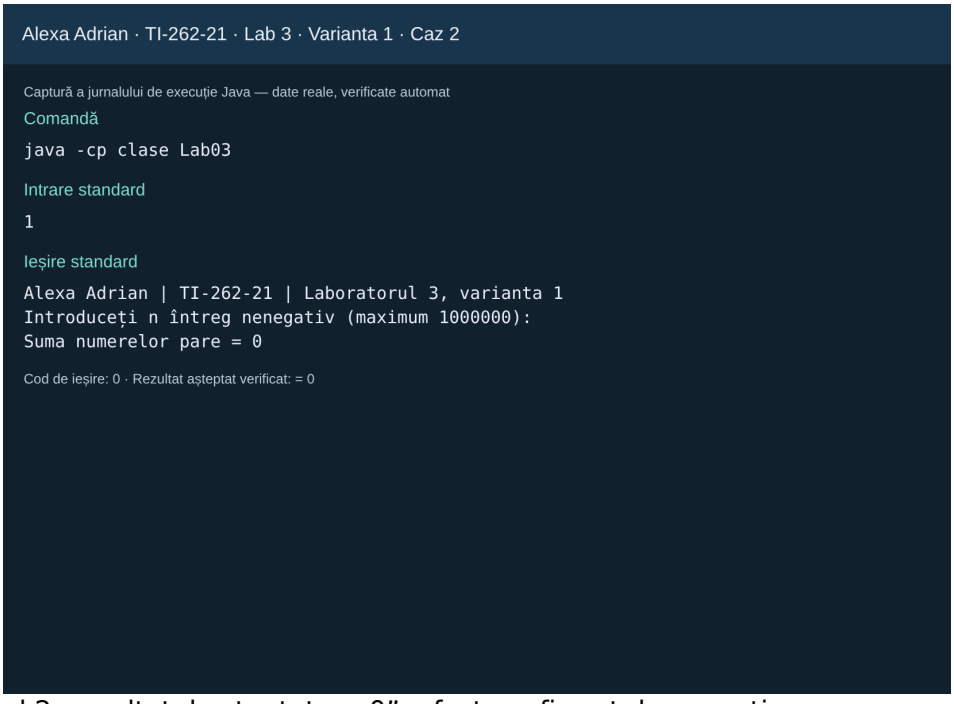


Figura 2. Cazul 2: rezultatul așteptat „= 0” a fost confirmat de execuție.

Capturi de ecran și interpretare



Figura 3. Cazul 3: rezultatul așteptat „Eroare” a fost confirmat de execuție.

Concluzii

1. Pasul 2 parcurge exclusiv numerele pare și simplifică bucla.

2. Formula k(k + 1) confirmă independent suma obținută iterativ.

3. BigInteger previne depășirea numerică observabilă pentru valori mari ale lui n.

Întrebări pentru pregătirea susținerii

• De ce suma inițială este zero?

• Ce rezultat avem pentru n = 1?

• De ce reatribuim suma.add(...) ?

Reproducerea lucrării

Din rădăcina arhivei: javac -encoding UTF-8 -d clase cod/Lab03.java, apoi java -cp clase Lab03. Este necesar un JDK 21 sau mai nou. Pentru Lab09, un argument opțional stabilește calea fișierului TSV; fără argument se folosește studenti.tsv în directorul curent.

Surse

1. „Lucrările de laborator la disciplina Bazele Programării în Java”, anul 2024–2025, actualizare 16.09.2024, PDF furnizat pentru această lucrare.

2. Documentația Java SE 21: https://docs.oracle.com/en/java/javase/21/docs/api/ — clasele Scanner, Math, String, Files și BigInteger, după caz.

Varianta electronică include sursa integrală, capturile PNG, jurnalele TXT, fișierele de date și instrucțiunile de rulare. Datele de prezentare, nota și semnătura se completează la susținere. Emblema de pe copertă este o reproducere grafică după imaginea furnizată de student.

MINISTERUL EDUCAȚIEI ȘI CERCETĂRII AL REPUBLICII MOLDOVA

UNIVERSITATEA LIBERĂ INTERNAȚIONALĂ DIN MOLDOVA

ULIM



Facultatea Informatică, Inginerie și Design

Catedra Tehnologii Informaționale

RAPORT

Lucrarea de laborator nr. 4 · Varianta 1

Disciplina: Bazele Programării în Java

Tema: Tablouri unidimensionale (vectori)

Anul de studii al cerințelor: 2024–2025

A elaborat: Alexa Adrian · Grupa TI-262-21

Profesor: Mititelu Vitalii

Data prezentării: _____

Nota: _____ Semnătura profesorului: _____

Chișinău

Scopul și sarcinile lucrării

Calcularea mediei aritmetice a elementelor unui vector.

Material teoretic

Un vector Java este o colecție de elemente de același tip, accesate prin indici. Pentru un tablou de lungime n, indicii valizi sunt 0, 1, …, n − 1. Lungimea este disponibilă prin proprietatea length. Accesarea unui indice din afara limitelor produce o excepție, de aceea bucla de citire folosește condiția i < n.

Media aritmetică este suma valorilor împărțită la numărul acestora. Pentru un vector nevid v, formula este M = (v[0] + … + v[n − 1]) / n. Nu este necesar ca elementele să fie pozitive; o medie poate fi negativă sau zero. Un vector gol nu are medie aritmetică definită în această lucrare.

Programul folosește double[] pentru a accepta valori reale. Suma se inițializează cu 0 și se calculează printr-o buclă for-each, care parcurge toate elementele fără gestionarea explicită a indicelui. Citirea rămâne indexată pentru a putea atribui fiecare poziție.

Se validează întâi dimensiunea, apoi elementele. Lungimea este limitată la 10000, iar valorile finite au modulul de cel mult 10^{100}. Aceste limite păstrează suma în domeniul finit al lui double. Afișarea cu șase zecimale reprezintă o rotunjire a rezultatului, nu o precizie matematică absolută. Anularea numerică poate afecta ultimele cifre pentru date cu ordine de mărime foarte diferite.

Convenții de lucru: surse în UTF-8, Java 21, intrare de la tastatură și mesaje în limba română. Separatorul zecimal la tastare este punctul. Cazurile de test au fost executate efectiv; jurnalele brute și codul sunt incluse în varianta electronică.

Algoritmul și verificarea rezultatului

1. Se citește n și se respinge n ≤ 0 sau n > 10000. 2. Se alocă vectorul. 3. Se citesc și se validează elementele. 4. Se însumează și se împarte suma la n.

Pentru [2, 4, 6, 8], suma este 20, n este 4 și media este 5. Pentru [−3, 0, 3], suma și media sunt zero. Pentru n = 0 se raportează eroare înaintea alocării și împărțirii.

Complexitate: O(n) timp și O(n) memorie pentru vector. Deși media ar putea fi calculată direct în timpul citirii, stocarea explicită respectă tema tablourilor unidimensionale.

Planul de testare și rezultatele

Cazul 1: 4 | 2 4 6 8. Verificare: „= 5.000000”. Rezultat: trecut; cod de ieșire 0.

Cazul 2: 3 | -3 0 3. Verificare: „= 0.000000”. Rezultat: trecut; cod de ieșire 0.

Cazul 3: 0. Verificare: „Eroare”. Rezultat: trecut; cod de ieșire 0.

Cazul 4: 1 | -7. Verificare: „= -7.000000”. Rezultat: trecut; cod de ieșire 0.

Cazul 5: 2 | NaN 1. Verificare: „Eroare”. Rezultat: trecut; cod de ieșire 0.

Pentru fiecare caz sunt păstrate separat intrarea, ieșirea și rezultatul așteptat. Capturile reproduc jurnalele reale într-o pagină de vizualizare; nu simulează un IDE. La lucrarea 9, capturile arată finalul jurnalului, iar fișierele TXT păstrează sesiunea integrală.

Codul integral — Lab04.java

```
import java.util.Locale;
import java.util.Scanner;

public class Lab04 {
    public static void main(String[] args) {
        Locale.setDefault(Locale.ROOT);
        Scanner in = new Scanner(System.in);
        System.out.println("Alexa Adrian | TI-262-21 | Laboratorul 4, varianta 1");
        System.out.println("Introduceți dimensiunea vectorului (1..10000):");
        if (!in.hasNextInt()) {
            System.out.println("Eroare: dimensiunea trebuie să fie întreagă.");
            return;
        }
        int n = in.nextInt();
        if (n < 1 || n > 10000) {
            System.out.println("Eroare: dimensiunea trebuie să fie în [1, 10000].");
            return;
        }
        double[] v = new double[n];
        System.out.println("Introduceți cele " + n + " elemente reale:");
        for (int i = 0; i < n; i++) {
            if (!in.hasNextDouble()) {
                System.out.println("Eroare: element numeric lipsă sau invalid.");
                return;
            }
            v[i] = in.nextDouble();
            if (!Double.isFinite(v[i]) || Math.abs(v[i]) > 1e100) {
                System.out.println("Eroare: elementele trebuie să fie finite, |x| <=
    1e100.");
                return;
            }
        }
        double suma = 0;
        for (double x : v) {
            suma += x;
        }
        System.out.printf("Media aritmetică = %.6f%n", suma / n);
    }
}
```

Capturi de ecran și interpretare

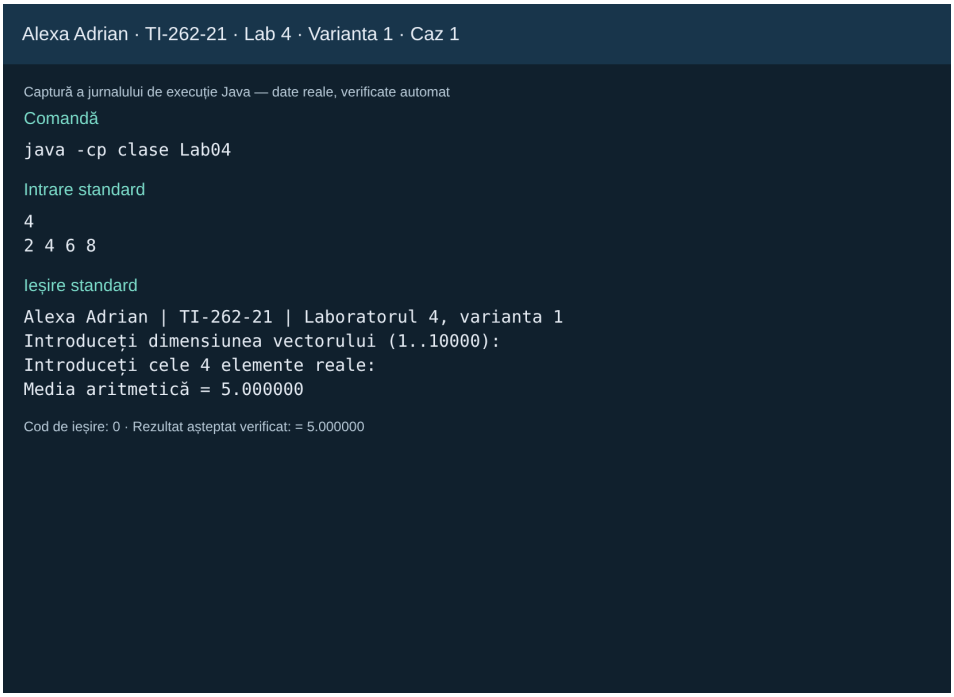


Figura 1. Cazul 1: rezultatul așteptat „= 5.000000” a fost confirmat de execuție.

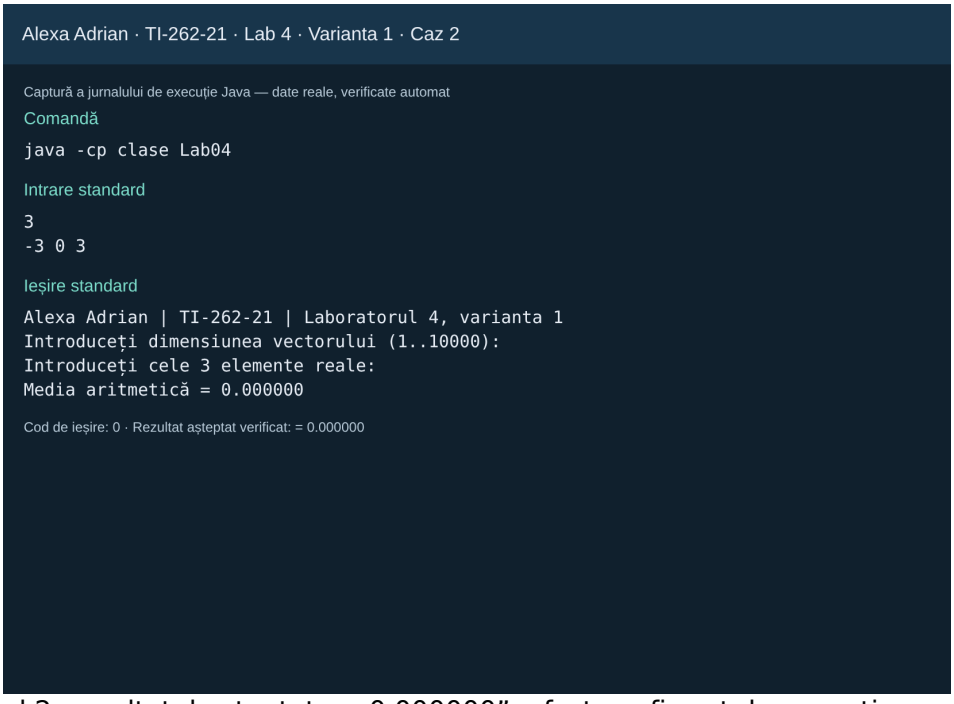


Figura 2. Cazul 2: rezultatul așteptat „= 0.000000” a fost confirmat de execuție.

Capturi de ecran și interpretare



Figura 3. Cazul 3: rezultatul așteptat „Eroare” a fost confirmat de execuție.

Concluzii

1. Vectorul stochează explicit datele și permite parcurgerea lor uniformă.

2. Validarea dimensiunii previne împărțirea la zero pentru un vector gol.

3. Media calculată coincide cu rezultatele manuale pentru valori pozitive și mixte.

Întrebări pentru pregătirea susținerii

• De ce ultimul indice este n − 1?

• De ce vectorul gol este respins?

• Ce rol are for-each?

Reproducerea lucrării

Din rădăcina arhivei: javac -encoding UTF-8 -d clase cod/Lab04.java, apoi java -cp clase Lab04. Este necesar un JDK 21 sau mai nou. Pentru Lab09, un argument opțional stabilește calea fișierului TSV; fără argument se folosește studenti.tsv în directorul curent.

Surse

1. „Lucrările de laborator la disciplina Bazele Programării în Java”, anul 2024–2025, actualizare 16.09.2024, PDF furnizat pentru această lucrare.

2. Documentația Java SE 21: https://docs.oracle.com/en/java/javase/21/docs/api/ — clasele Scanner, Math, String, Files și BigInteger, după caz.

Varianta electronică include sursa integrală, capturile PNG, jurnalele TXT, fișierele de date și instrucțiunile de rulare. Datele de prezentare, nota și semnătura se completează la susținere. Emblema de pe copertă este o reproducere grafică după imaginea furnizată de student.

MINISTERUL EDUCAȚIEI ȘI CERCETĂRII AL REPUBLICII MOLDOVA

UNIVERSITATEA LIBERĂ INTERNAȚIONALĂ DIN MOLDOVA

ULIM



Facultatea Informatică, Inginerie și Design

Catedra Tehnologii Informaționale

RAPORT

Lucrarea de laborator nr. 5 · Varianta 1

Disciplina: Bazele Programării în Java

Tema: Tablouri bidimensionale (matrice)

Anul de studii al cerințelor: 2024–2025

A elaborat: Alexa Adrian · Grupa TI-262-21

Profesor: Mititelu Vitalii

Data prezentării: _____

Nota: _____ Semnătura profesorului: _____

Chișinău

Scopul și sarcinile lucrării

Calcularea mediei aritmetice a tuturor elementelor unei matrice.

Material teoretic

O matrice este reprezentată în Java printr-un tablou de tablouri. Pentru double[n][m], primul indice selectează rândul, iar al doilea selectează coloana. În această lucrare se alocă o matrice dreptunghiulară; nu este necesar ca n și m să fie egale, deoarece varianta 1 nu operează asupra diagonalelor.

Parcurgerea completă folosește două cicluri imbricate. Ciclul exterior fixează rândul, iar cel interior vizitează toate coloanele acestuia. Datele sunt citite în ordinea rândurilor, de la stânga la dreapta. Această convenție este precizată în mesajul de intrare pentru a evita confuzia asupra pozițiilor.

Media aritmetică a matricei este suma tuturor valorilor împărțită la $n \cdot m$. Suma fiecărui element este adăugată o singură dată. Nu calculăm media mediilor unor rânduri cu lungimi diferite; matricea folosită este dreptunghiulară, iar formula globală exprimă direct cerința.

Ambele dimensiuni trebuie să fie pozitive și să nu depășească 100. Astfel, numărul elementelor este cel mult 10000 și produsul $n \cdot m$ nu poate depăși int. Fiecare element trebuie să fie finit și să aibă modulul cel mult 10^{100}. Tipul double permite fracții, însă introduce rotunjiri. Scanner folosește punctul ca separator zecimal prin Locale.ROOT; virgula zecimală din notația românească trebuie înlocuită cu punct la tastare.

Convenții de lucru: surse în UTF-8, Java 21, intrare de la tastatură și mesaje în limba română. Separatorul zecimal la tastare este punctul. Cazurile de test au fost executate efectiv; jurnalele brute și codul sunt incluse în varianta electronică.

Algoritmul și verificarea rezultatului

Se verifică separat rândurile și coloanele, apoi se alocă matricea. Pentru fiecare pereche (i, j), elementul este citit, validat, memorat și adăugat în sumă. Rezultatul se obține prin împărțirea la numărul total de elemente.

Exemplul 2 × 3 cu rândurile [1, 2, 3] și [4, 5, 6] are suma 21 și media 21 / 6 = 3,5. Matricea 1 × 1 cu valoarea −7 are media −7. Dimensiunea 0 × 3 este respinsă.

Complexitate: O(n · m) timp și O(n · m) memorie. Verificarea pe o matrice dreptunghiulară demonstrează că programul nu impune nejustificat forma pătratică.

Planul de testare și rezultatele

Cazul 1: 2 3 | 1 2 3 4 5 6. Verificare: „= 3.500000”. Rezultat: trecut; cod de ieșire 0.

Cazul 2: 1 1 | -7. Verificare: „= -7.000000”. Rezultat: trecut; cod de ieșire 0.

Cazul 3: 0 3. Verificare: „Eroare”. Rezultat: trecut; cod de ieșire 0.

Cazul 4: 2 2 | -1 1 -2 2. Verificare: „= 0.000000”. Rezultat: trecut; cod de ieșire 0.

Pentru fiecare caz sunt păstrate separat intrarea, ieșirea și rezultatul așteptat. Capturile reproduc jurnalele reale într-o pagină de vizualizare; nu simulează un IDE. La lucrarea 9, capturile arată finalul jurnalului, iar fișierele TXT păstrează sesiunea integrală.

Codul integral — Lab05.java

```
import java.util.Locale;
import java.util.Scanner;

public class Lab05 {
    public static void main(String[] args) {
        Locale.setDefault(Locale.ROOT);
        Scanner in = new Scanner(System.in);
        System.out.println("Alexa Adrian | TI-262-21 | Laboratorul 5, varianta 1");
        System.out.println("Introduceți numărul de rânduri și de coloane (1..100):");
        if (!in.hasNextInt()) {
            System.out.println("Eroare: număr de rânduri invalid.");
            return;
        }
        int n = in.nextInt();
        if (!in.hasNextInt()) {
            System.out.println("Eroare: număr de coloane invalid.");
            return;
        }
        int m = in.nextInt();
        if (n < 1 || m < 1 || n > 100 || m > 100) {
            System.out.println("Eroare: dimensiunile trebuie să fie în [1, 100].");
            return;
        }
        double[][] a = new double[n][m];
        double suma = 0;
        System.out.println("Introduceți elementele pe rânduri:");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                if (!in.hasNextDouble()) {
                    System.out.println("Eroare: element numeric lipsă sau invalid.");
                    return;
                }
                a[i][j] = in.nextDouble();
                if (!Double.isFinite(a[i][j]) || Math.abs(a[i][j]) > 1e100) {
                    System.out.println("Eroare: elementele trebuie să fie finite, |x| <=
    1e100.");
                    return;
                }
                suma += a[i][j];
            }
        }
        System.out.printf("Media aritmetică = %.6f%n", suma / (n * m));
    }
}
```

Capturi de ecran și interpretare

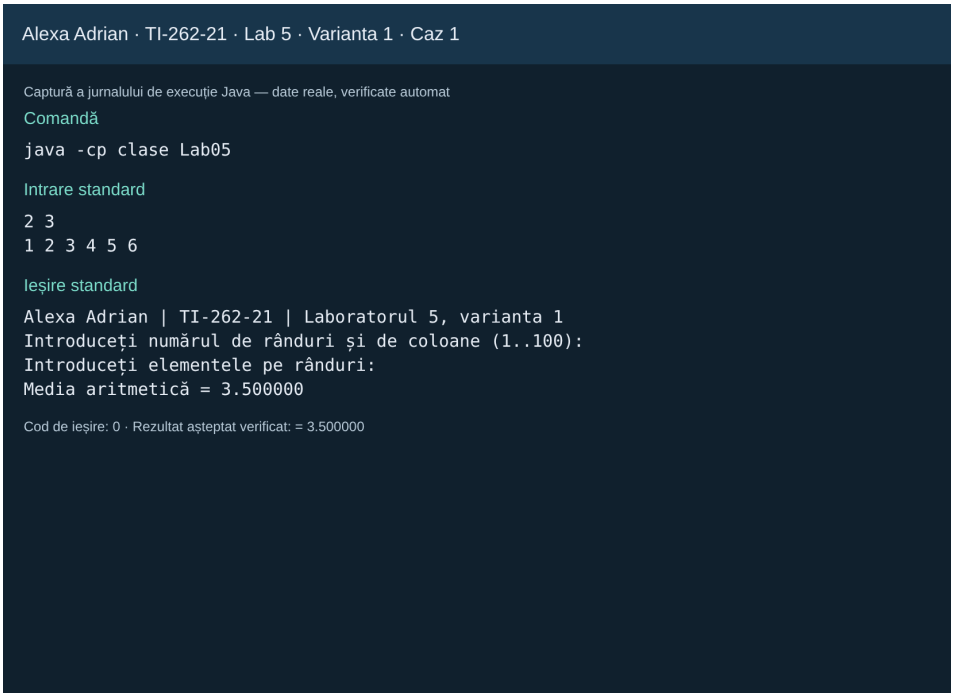


Figura 1. Cazul 1: rezultatul așteptat „= 3.500000” a fost confirmat de execuție.



Figura 2. Cazul 2: rezultatul așteptat „= -7.000000” a fost confirmat de execuție.

Capturi de ecran și interpretare

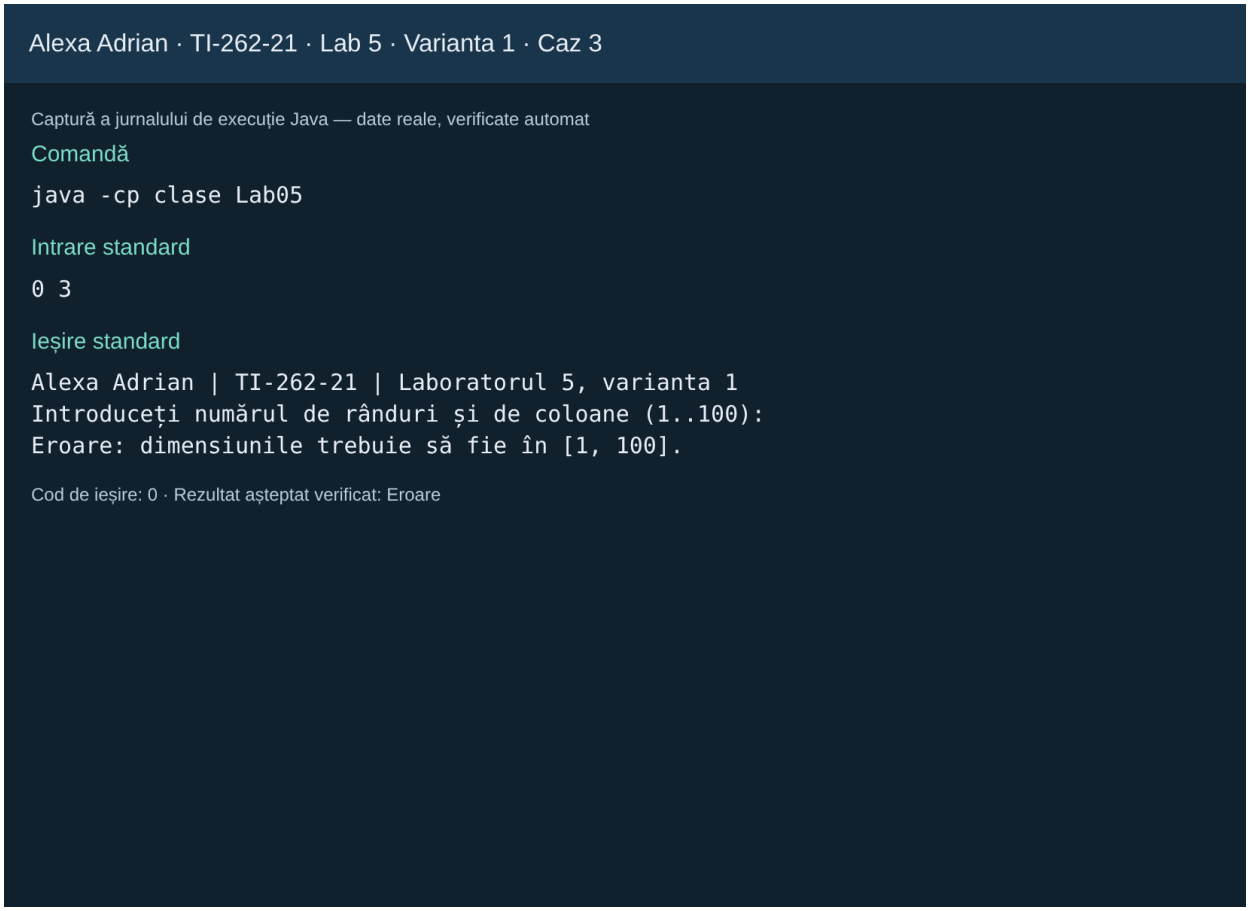


Figura 3. Cazul 3: rezultatul așteptat „Eroare” a fost confirmat de execuție.

Concluzii

1. Două cicluri imbricate vizitează toate elementele matricei o singură dată.

2. Media folosește n · m elemente, nu doar numărul rândurilor.

3. Testul dreptunghiular confirmă respectarea cerinței pentru varianta 1.

Întrebări pentru pregătirea susținerii

• Ce reprezintă a[i][j]?

• De ce împărțim la n · m?

• Trebuie matricea să fie pătratică?

Reproducerea lucrării

Din rădăcina arhivei: javac -encoding UTF-8 -d clase cod/Lab05.java, apoi java -cp clase Lab05. Este necesar un JDK 21 sau mai nou. Pentru Lab09, un argument opțional stabilește calea fișierului TSV; fără argument se folosește studenti.tsv în directorul curent.

Surse

1. „Lucrările de laborator la disciplina Bazele Programării în Java”, anul 2024–2025, actualizare 16.09.2024, PDF furnizat pentru această lucrare.

2. Documentația Java SE 21: https://docs.oracle.com/en/java/javase/21/docs/api/ — clasele Scanner, Math, String, Files și BigInteger, după caz.

Varianta electronică include sursa integrală, capturile PNG, jurnalele TXT, fișierele de date și instrucțiunile de rulare. Datele de prezentare, nota și semnătura se completează la susținere. Emblema de pe copertă este o reproducere grafică după imaginea furnizată de student.

MINISTERUL EDUCAȚIEI ȘI CERCETĂRII AL REPUBLICII MOLDOVA

UNIVERSITATEA LIBERĂ INTERNAȚIONALĂ DIN MOLDOVA

ULIM



Facultatea Informatică, Inginerie și Design

Catedra Tehnologii Informaționale

RAPORT

Lucrarea de laborator nr. 6 · Varianta 1

Disciplina: Bazele Programării în Java

Tema: Șiruri de caractere (String-uri)

Anul de studii al cerințelor: 2024–2025

A elaborat: Alexa Adrian · Grupa TI-262-21

Profesor: Mititelu Vitalii

Data prezentării: _____

Nota: _____ Semnătura profesorului: _____

Chișinău

Scopul și sarcinile lucrării

Determinarea prezenței literei „a” într-un șir de caractere, cu oprirea căutării la prima apariție.

Material teoretic

Clasa String reprezintă șiruri de caractere. Obiectele String sunt imuabile: operațiile care par să modifice textul creează, în general, un șir nou. Pentru simpla căutare a unui caracter nu este necesară copierea sau transformarea întregului text.

Metoda indexOf('a') întoarce poziția primei apariții sau valoarea −1 dacă litera nu există. Testul indexOf('a') >= 0 exprimă direct întrebarea din cerință. Căutarea se oprește la prima apariție; numărarea tuturor aparițiilor ar efectua operații inutile atunci când se cere doar prezența.

Citirea se face cu nextLine, care păstrează spațiile din interiorul liniei. Metoda next ar citi numai primul cuvânt și ar putea omite o literă aflată mai târziu în text. O linie goală este o intrare validă și nu conține litera căutată. Lipsa completă a unei linii la sfârșitul fluxului este tratată separat.

Interpretarea aleasă este literală: litera latină mică a este diferită de A, ă și â. Cerința nu solicită ignorarea majusculelor sau a diacriticelor. Java folosește UTF-16 pentru String, iar pentru caracterul a reprezentarea ocupă o singură unitate char. Testele cu text românesc nu înlocuiesc diacriticele și nu normalizează în mod ascuns intrarea.

Convenții de lucru: surse în UTF-8, Java 21, intrare de la tastatură și mesaje în limba română. Separatorul zecimal la tastare este punctul. Cazurile de test au fost executate efectiv; jurnalele brute și codul sunt incluse în varianta electronică.

Algoritmul și verificarea rezultatului

Programul citește întreaga linie și apelează indexOf o singură dată. Dacă rezultatul este nenegativ, afișează că litera este prezentă; în caz contrar, afișează absența.

Textul „Java” conține a la indicele 1. „ULIM” nu conține a. Șirul gol nu conține a. Testul suplimentar „AĂÂ” confirmă diferența dintre caracterele distincte.

Complexitate: în cel mai nefavorabil caz O(L), unde L este lungimea șirului. Pentru o apariție la început, căutarea se termină imediat. Memoria suplimentară a operației de căutare este O(1), pe lângă memoria textului.

Planul de testare și rezultatele

Cazul 1: Java. Verificare: „este prezentă.”. Rezultat: trecut; cod de ieșire 0.

Cazul 2: ULIM. Verificare: „nu este prezentă.”. Rezultat: trecut; cod de ieșire 0.

Cazul 3: linie goală. Verificare: „nu este prezentă.”. Rezultat: trecut; cod de ieșire 0.

Cazul 4: AĂÂ. Verificare: „nu este prezentă.”. Rezultat: trecut; cod de ieșire 0.

Cazul 5: a. Verificare: „este prezentă.”. Rezultat: trecut; cod de ieșire 0.

Pentru fiecare caz sunt păstrate separat intrarea, ieșirea și rezultatul așteptat. Capturile reproduc jurnalele reale într-o pagină de vizualizare; nu simulează un IDE. La lucrarea 9, capturile arată finalul jurnalului, iar fișierele TXT păstrează sesiunea integrală.

Codul integral — Lab06.java

```
import java.util.Scanner;

public class Lab06 {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.println("Alexa Adrian | TI-262-21 | Laboratorul 6, varianta 1");
        System.out.println("Introduceți un șir de caractere:");
        if (!in.hasNextLine()) {
            System.out.println("Eroare: nu a fost transmisă nicio linie.");
            return;
        }
        String text = in.nextLine();
        // Căutarea se oprește la prima apariție; nu numărăm toate aparițiile.
        if (text.indexOf('a') >= 0) {
            System.out.println("Litera «a» este prezentă.");
        } else {
            System.out.println("Litera «a» nu este prezentă.");
        }
    }
}
```

Capturi de ecran și interpretare

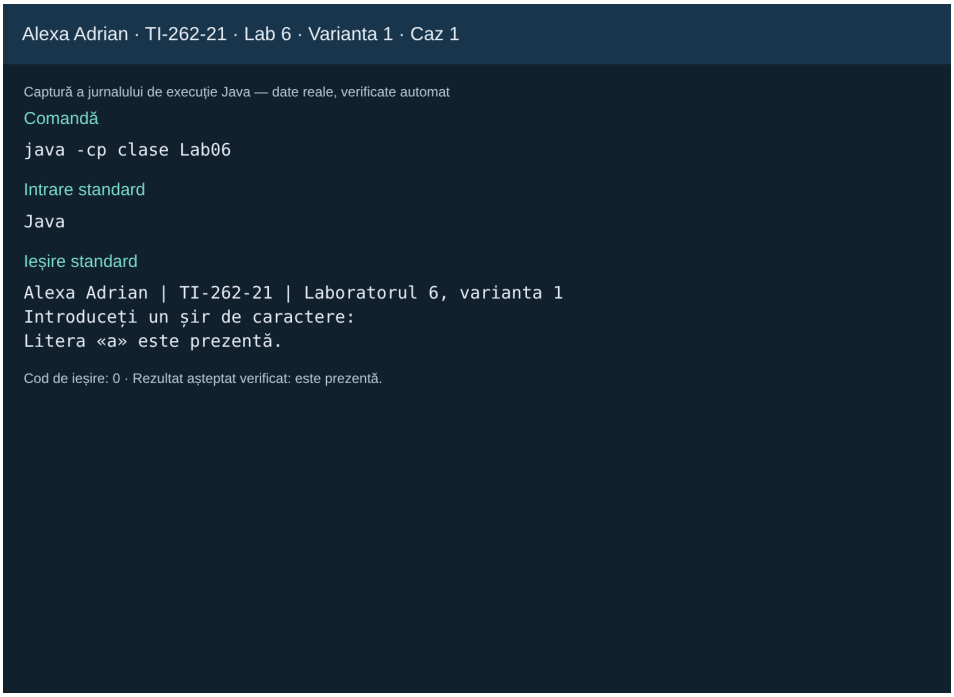


Figura 1. Cazul 1: rezultatul așteptat „este prezentă.” a fost confirmat de execuție.

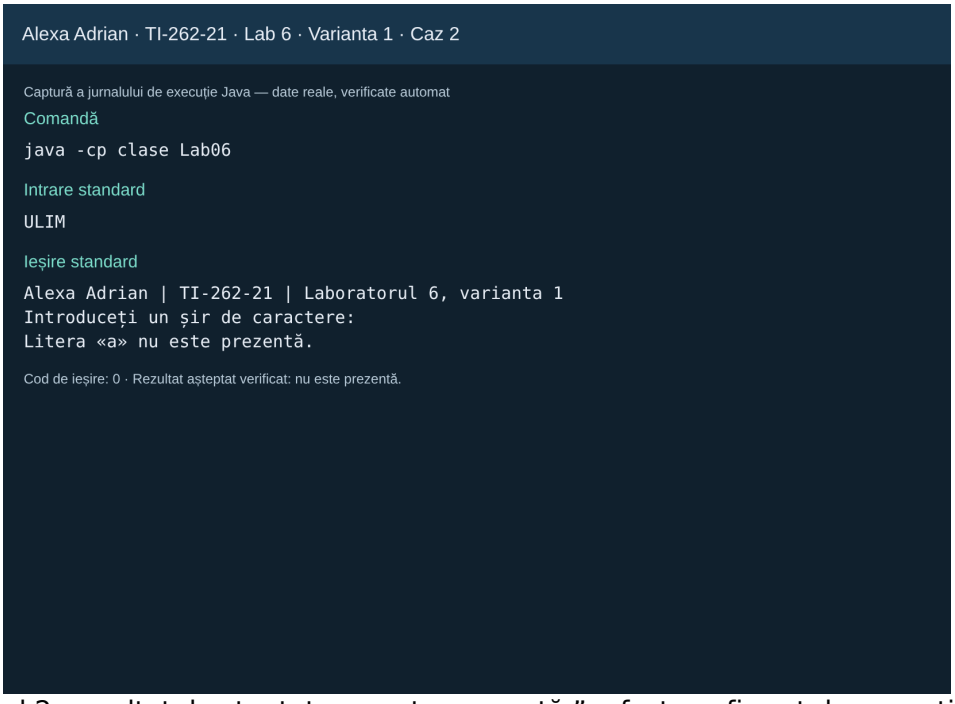


Figura 2. Cazul 2: rezultatul așteptat „nu este prezentă.” a fost confirmat de execuție.

Capturi de ecran și interpretare

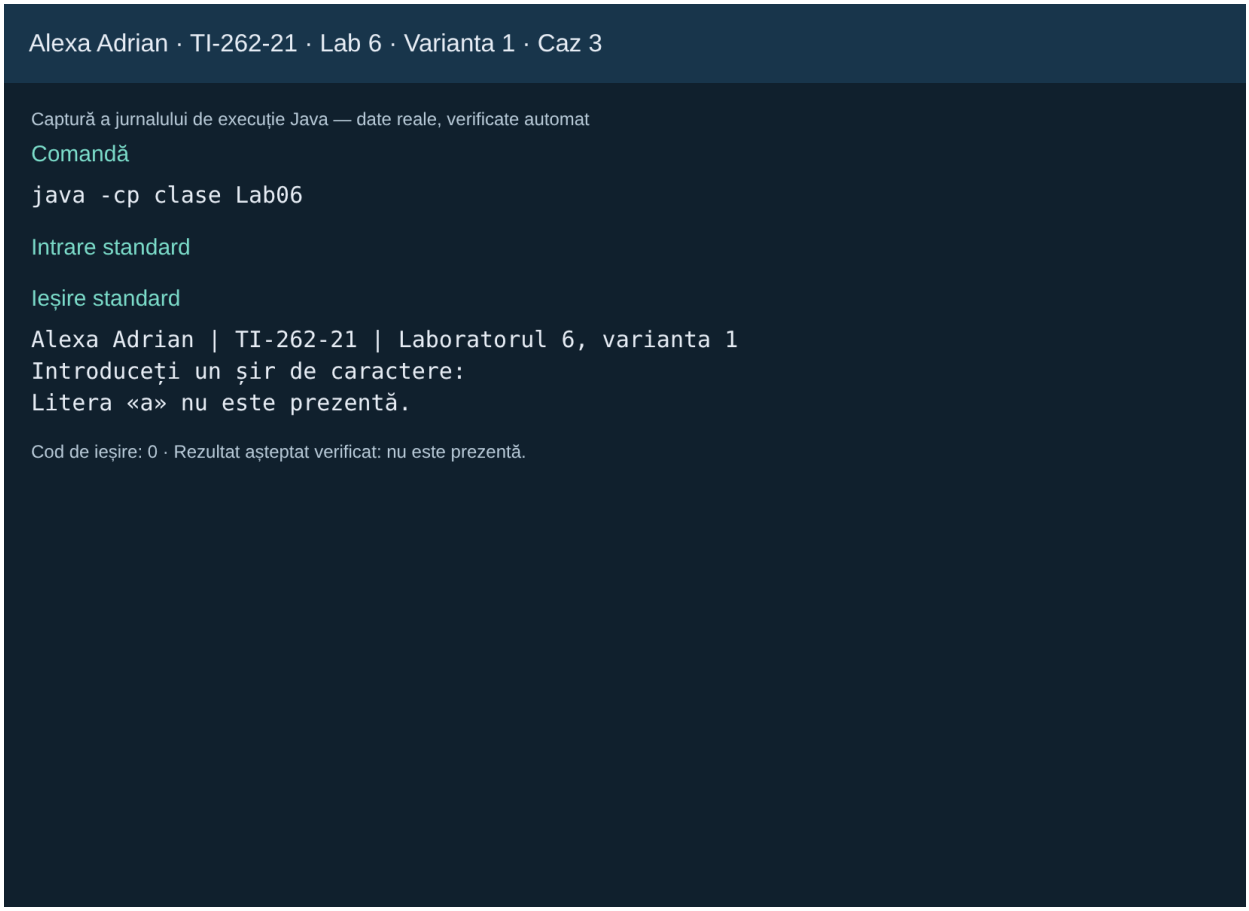


Figura 3. Cazul 3: rezultatul așteptat „nu este prezentă.” a fost confirmat de execuție.

Concluzii

1. indexOf oferă direct testul de prezență și evită numărarea inutilă.

2. nextLine permite verificarea întregii linii, inclusiv a textului cu spații.

3. Șirul gol, majusculele și diacriticele necesită o interpretare explicită.

Întrebări pentru pregătirea susținerii

• Ce semnifică valoarea −1?

• Care este diferența dintre next și nextLine?

• De ce A nu este a?

Reproducerea lucrării

Din rădăcina arhivei: javac -encoding UTF-8 -d clase cod/Lab06.java, apoi java -cp clase Lab06. Este necesar un JDK 21 sau mai nou. Pentru Lab09, un argument opțional stabilește calea fișierului TSV; fără argument se folosește studenti.tsv în directorul curent.

Surse

1. „Lucrările de laborator la disciplina Bazele Programării în Java”, anul 2024–2025, actualizare 16.09.2024, PDF furnizat pentru această lucrare.

2. Documentația Java SE 21: https://docs.oracle.com/en/java/javase/21/docs/api/ — clasele Scanner, Math, String, Files și BigInteger, după caz.

Varianta electronică include sursa integrală, capturile PNG, jurnalele TXT, fișierele de date și instrucțiunile de rulare. Datele de prezentare, nota și semnătura se completează la susținere. Emblema de pe copertă este o reproducere grafică după imaginea furnizată de student.

MINISTERUL EDUCAȚIEI ȘI CERCETĂRII AL REPUBLICII MOLDOVA

UNIVERSITATEA LIBERĂ INTERNAȚIONALĂ DIN MOLDOVA

ULIM



Facultatea Informatică, Inginerie și Design

Catedra Tehnologii Informaționale

RAPORT

Lucrarea de laborator nr. 7 · Varianta 1

Disciplina: Bazele Programării în Java

Tema: Metode (funcții) care întorc valori

Anul de studii al cerințelor: 2024–2025

A elaborat: Alexa Adrian · Grupa TI-262-21

Profesor: Mititelu Vitalii

Data prezentării: _____

Nota: _____ Semnătura profesorului: _____

Chișinău

Scopul și sarcinile lucrării

Calcularea mediei aritmetice a trei numere întregi printr-o metodă proprie care întoarce o valoare.

Material teoretic

O metodă grupează instrucțiuni care efectuează o operație bine definită. Semnătura include numele și tipurile parametrilor, iar tipul întors precizează natura rezultatului. Metoda media primește trei int și întoarce un double. Instrucțiunea return transmite valoarea calculată apelantului.

Metoda este declarată static pentru a putea fi apelată din main fără crearea unui obiect. Parametrii a, b și c sunt variabile locale metodei. Java transmite valorile parametrilor primitivi prin valoare, deci calculul nu modifică datele din vectorul folosit la citire.

Expresia ((double) a + b + c) / 3.0 este aleasă atent. Conversia primului operand la double înainte de adunare determină evaluarea sumei în virgulă mobilă. Dacă am scrie (double) (a + b + c), adunarea s-ar efectua mai întâi în int și ar putea depăși intervalul acestui tip. Împărțirea la 3.0 păstrează partea fracționară.

Separarea dintre interfața de intrare și calcul simplifică verificarea. Main validează și citește datele, iar media efectuează numai operația matematică. O metodă care întoarce valoarea este reutilizabilă în alte contexte; o metodă care doar afișează nu ar satisface la fel de clar cerința acestei lucrări. Rezultatul se formatează la șase zecimale în main.

Convenții de lucru: surse în UTF-8, Java 21, intrare de la tastatură și mesaje în limba română. Separatorul zecimal la tastare este punctul. Cazurile de test au fost executate efectiv; jurnalele brute și codul sunt incluse în varianta electronică.

Algoritmul și verificarea rezultatului

Se citesc trei int, se apelează media(v[0], v[1], v[2]) și se afișează valoarea returnată. Pentru 1, 2, 3, rezultatul este 6 / 3 = 2. Pentru −3, 0, 3, media este zero.

Cazul de limită folosește trei valori 2147483647. Media lor trebuie să rămână 2147483647. Conversia înainte de adunare previne depășirea care ar apărea într-o sumă de tip int. Valorile int și suma a trei astfel de valori sunt reprezentabile exact în double; împărțirea poate introduce o rotunjire pentru fracții periodice.

Complexitate: O(1) timp și O(1) memorie. Metoda întoarce rezultatul, iar apelantul decide cum îl utilizează.

Planul de testare și rezultatele

Cazul 1: 1 2 3. Verificare: „= 2.000000”. Rezultat: trecut; cod de ieșire 0.

Cazul 2: -3 0 3. Verificare: „= 0.000000”. Rezultat: trecut; cod de ieșire 0.

Cazul 3: 2147483647 2147483647 2147483647. Verificare: „= 2147483647.000000”. Rezultat: trecut; cod de ieșire 0.

Cazul 4: x. Verificare: „Eroare”. Rezultat: trecut; cod de ieșire 0.

Pentru fiecare caz sunt păstrate separat intrarea, ieșirea și rezultatul așteptat. Capturile reproduc jurnalele reale într-o pagină de vizualizare; nu simulează un IDE. La lucrarea 9, capturile arată finalul jurnalului, iar fișierele TXT păstrează sesiunea integrală.

Codul integral — Lab07.java

```
import java.util.Locale;
import java.util.Scanner;

public class Lab07 {
    // Conversia înaintea adunării previne depășirea tipului int.
    public static double media(int a, int b, int c) {
        return ((double) a + b + c) / 3.0;
    }

    public static void main(String[] args) {
        Locale.setDefault(Locale.ROOT);
        Scanner in = new Scanner(System.in);
        System.out.println("Alexa Adrian | TI-262-21 | Laboratorul 7, varianta 1");
        System.out.println("Introduceți trei numere întregi (int):");
        int[] v = new int[3];
        for (int i = 0; i < 3; i++) {
            if (!in.hasNextInt()) {
                System.out.println("Eroare: sunt necesare trei valori de tip int.");
                return;
            }
            v[i] = in.nextInt();
        }
        System.out.printf("Media aritmetică = %.6f%n", media(v[0], v[1], v[2]));
    }
}
```

Capturi de ecran și interpretare

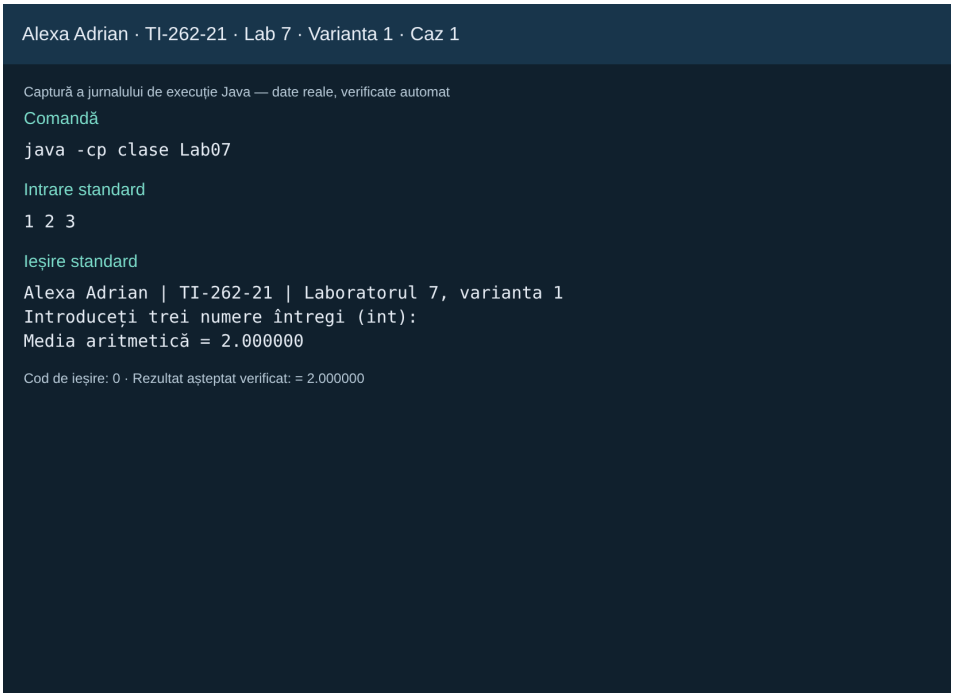


Figura 1. Cazul 1: rezultatul așteptat „= 2.000000” a fost confirmat de execuție.

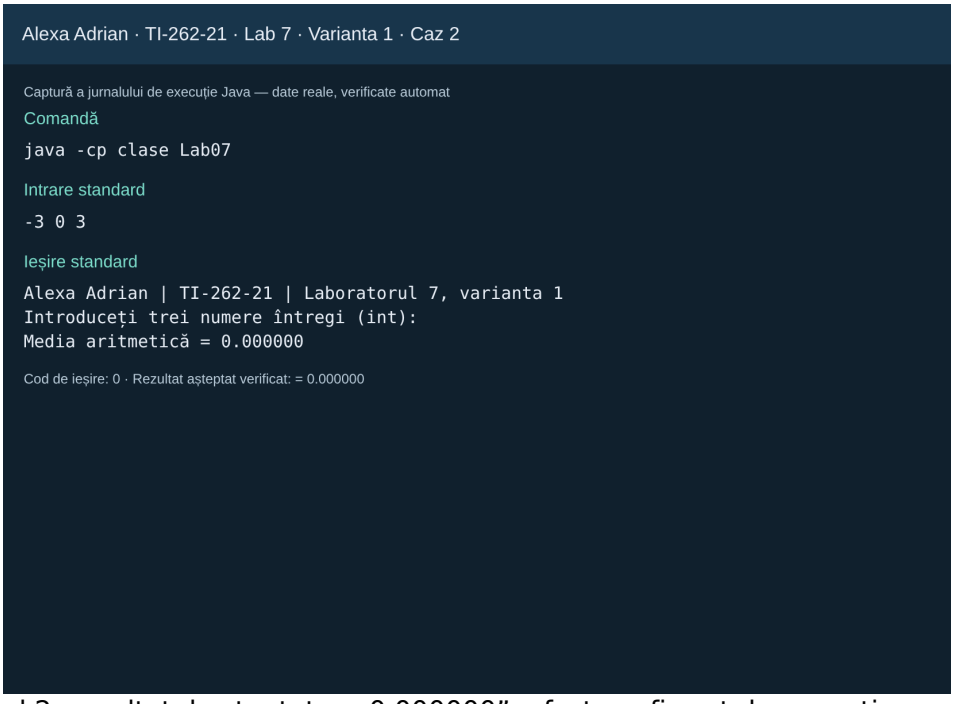


Figura 2. Cazul 2: rezultatul așteptat „= 0.000000” a fost confirmat de execuție.

Capturi de ecran și interpretare

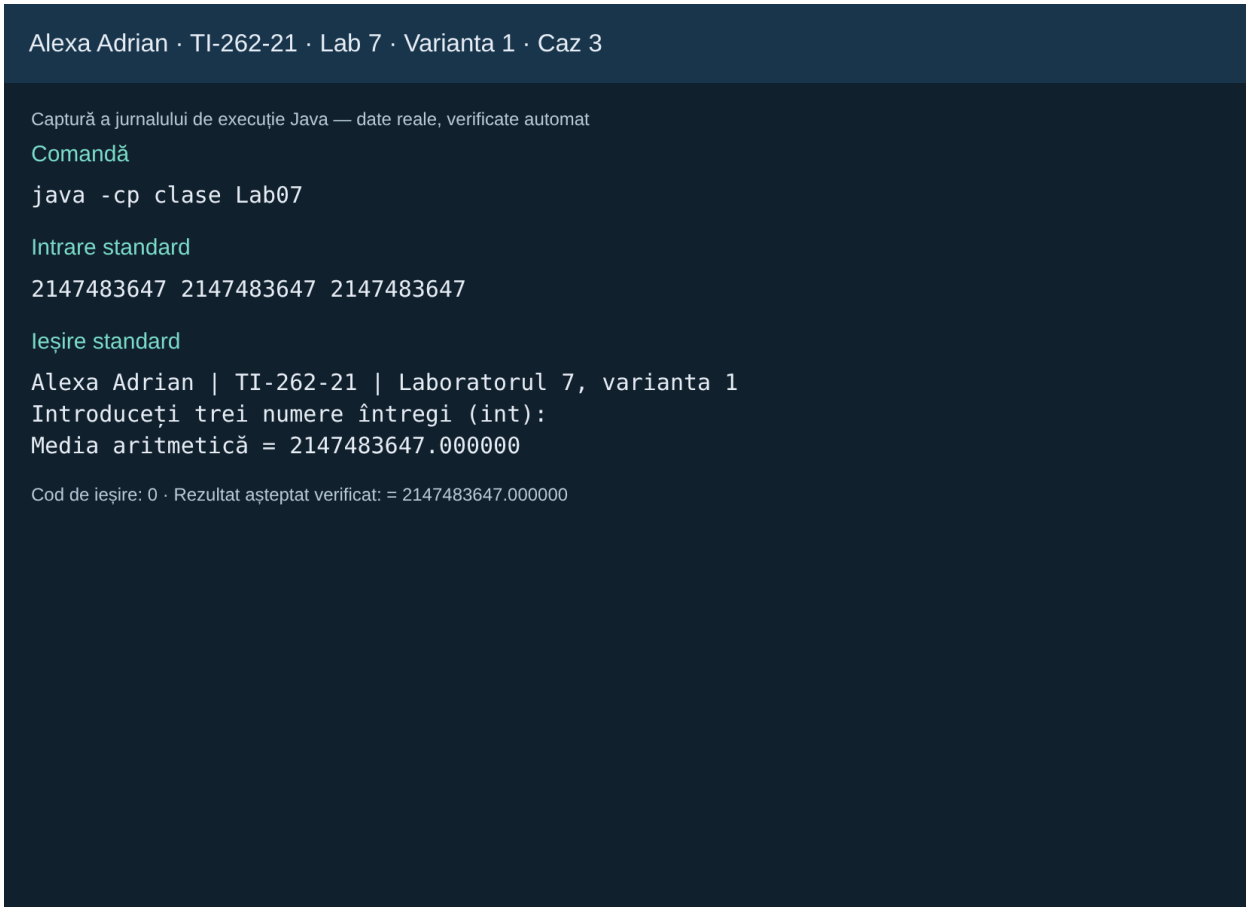


Figura 3. Cazul 3: rezultatul așteptat „= 2147483647.000000” a fost confirmat de execuție.

Concluzii

1. Metoda proprie media întoarce un rezultat și separă calculul de citire și afișare.

2. Conversia înainte de adunare previne depășirea tipului int.

3. Testul cu trei valori maxime confirmă comportamentul la limita datelor de intrare.

Întrebări pentru pregătirea susținerii

• Ce rol are return?

• De ce metoda este static?

• Unde trebuie plasată conversia la double?

Reproducerea lucrării

Din rădăcina arhivei: javac -encoding UTF-8 -d clase cod/Lab07.java, apoi java -cp clase Lab07. Este necesar un JDK 21 sau mai nou. Pentru Lab09, un argument opțional stabilește calea fișierului TSV; fără argument se folosește studenti.tsv în directorul curent.

Surse

1. „Lucrările de laborator la disciplina Bazele Programării în Java”, anul 2024–2025, actualizare 16.09.2024, PDF furnizat pentru această lucrare.

2. Documentația Java SE 21: https://docs.oracle.com/en/java/javase/21/docs/api/ — clasele Scanner, Math, String, Files și BigInteger, după caz.

Varianta electronică include sursa integrală, capturile PNG, jurnalele TXT, fișierele de date și instrucțiunile de rulare. Datele de prezentare, nota și semnătura se completează la susținere. Emblema de pe copertă este o reproducere grafică după imaginea furnizată de student.

MINISTERUL EDUCAȚIEI ȘI CERCETĂRII AL REPUBLICII MOLDOVA

UNIVERSITATEA LIBERĂ INTERNAȚIONALĂ DIN MOLDOVA

ULIM



Facultatea Informatică, Inginerie și Design

Catedra Tehnologii Informaționale

RAPORT

Lucrarea de laborator nr. 8 · Varianta 1

Disciplina: Bazele Programării în Java

Tema: Clase

Anul de studii al cerințelor: 2024–2025

A elaborat: Alexa Adrian · Grupa TI-262-21

Profesor: Mititelu Vitalii

Data prezentării: _____

Nota: _____ Semnătura profesorului: _____

Chișinău

Scopul și sarcinile lucrării

Crearea unei clase pentru citirea, afișarea și calcularea mediei unui tablou; utilizarea obiectului din altă clasă.

Material teoretic

O clasă definește structura și comportamentul unor obiecte. Câmpurile păstrează starea, iar metodele oferă operațiile permise. În această lucrare, clasa Tablou încapsulează vectorul de numere reale și metodele citeste, afiseaza și media. Clasa Lab08 coordonează execuția și creează un obiect prin new Tablou().

Câmpul elemente este private pentru a limita accesul direct din exterior. Metodele publice reprezintă interfața clasei. În contrast cu metoda static din lucrarea precedentă, metodele clasei Tablou lucrează asupra datelor obiectului curent. Fiecare obiect ar putea păstra propriul vector.

Metoda citeste primește Scanner ca parametru; astfel, clasa nu creează un al doilea cititor asupra aceluiași flux. Metoda verifică dimensiunea și fiecare valoare înainte ca fluxul normal să ajungă la calculul mediei. Datele nevalide produc IllegalArgumentException, iar main interceptează excepția și afișează un mesaj clar.

Ordinea apelurilor este parte a contractului: citeste trebuie să se încheie cu succes înainte de afiseaza și media. Metoda media presupune un vector inițializat și nevid; ea nu este o interfață generală pentru apeluri în orice ordine. Arrays.toString produce o reprezentare lizibilă a tuturor elementelor. Cele două clase sunt incluse în același fișier sursă, însă numai Lab08 este publică; această organizare este legală în Java.

Convenții de lucru: surse în UTF-8, Java 21, intrare de la tastatură și mesaje în limba română. Separatorul zecimal la tastare este punctul. Cazurile de test au fost executate efectiv; jurnalele brute și codul sunt incluse în varianta electronică.

Algoritmul și verificarea rezultatului

Lab08 construiește obiectul, apelează citirea, afișează tabloul și folosește rezultatul întors de media. Dacă citirea eșuează, excepția întrerupe secvența și calculul nu este executat.

Pentru [2, 4, 6, 8], obiectul păstrează patru elemente și întoarce 5. Pentru un singur element −2, media este −2. Pentru dimensiunea zero, apelul de citire aruncă excepția, iar main afișează eroarea.

Complexitate: O(n) timp pentru citire, afișare și medie; O(n) memorie pentru câmpul vector. Cerința a două clase este îndeplinită prin Tablou și Lab08, fără a impune două fișiere.

Planul de testare și rezultatele

Cazul 1: 4 | 2 4 6 8. Verificare: „= 5.000000”. Rezultat: trecut; cod de ieșire 0.

Cazul 2: 1 | -2. Verificare: „= -2.000000”. Rezultat: trecut; cod de ieșire 0.

Cazul 3: 0. Verificare: „Eroare”. Rezultat: trecut; cod de ieșire 0.

Cazul 4: 3 | -3 0 3. Verificare: „= 0.000000”. Rezultat: trecut; cod de ieșire 0.

Pentru fiecare caz sunt păstrate separat intrarea, ieșirea și rezultatul așteptat. Capturile reproduc jurnalele reale într-o pagină de vizualizare; nu simulează un IDE. La lucrarea 9, capturile arată finalul jurnalului, iar fișierele TXT păstrează sesiunea integrală.

Codul integral — Lab08.java

```
import java.util.Arrays;
import java.util.Locale;
import java.util.Scanner;

class Tablou {
    private double[] elemente;

    public void citeste(Scanner in) {
        System.out.println("Introduceți dimensiunea tabloului (1..10000):");
        if (!in.hasNextInt()) {
            throw new IllegalArgumentException("Dimensiune întreagă invalidă.");
        }
        int n = in.nextInt();
        if (n < 1 || n > 10000) {
            throw new IllegalArgumentException("Dimensiunea trebuie să fie în [1, 10000].");
        }
        elemente = new double[n];
        System.out.println("Introduceți elementele reale:");
        for (int i = 0; i < n; i++) {
            if (!in.hasNextDouble()) {
                throw new IllegalArgumentException("Element numeric lipsă sau invalid.");
            }
            elemente[i] = in.nextDouble();
            if (!Double.isFinite(elemente[i]) || Math.abs(elemente[i]) > 1e100) {
                throw new IllegalArgumentException("Element finit necesar, |x| <= 1e100.");
            }
        }
    }
```

Codul integral — Lab08.java (continuare)

```
    public double media() {
        double suma = 0;
        for (double x : elemente) {
            suma += x;
        }
        return suma / elemente.length;
    }

    public void afiseaza() {
        System.out.println("Tabloul: " + Arrays.toString(elemente));
    }
}

public class Lab08 {
    public static void main(String[] args) {
        Locale.setDefault(Locale.ROOT);
        Scanner in = new Scanner(System.in);
        System.out.println("Alexa Adrian | TI-262-21 | Laboratorul 8, varianta 1");
        Tablou tablou = new Tablou();
        try {
            tablou.citeste(in);
            tablou.afiseaza();
            System.out.printf("Media aritmetică = %.6f%n", tablou.media());
        } catch (IllegalArgumentException e) {
            System.out.println("Eroare: " + e.getMessage());
        }
    }
}
```

Capturi de ecran și interpretare

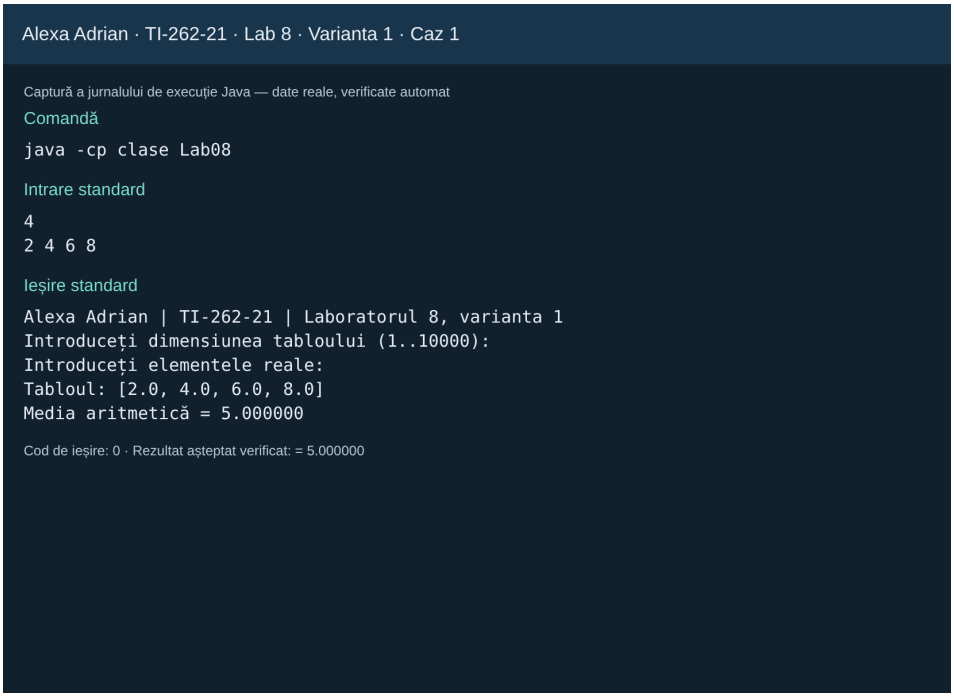


Figura 1. Cazul 1: rezultatul așteptat „= 5.000000” a fost confirmat de execuție.

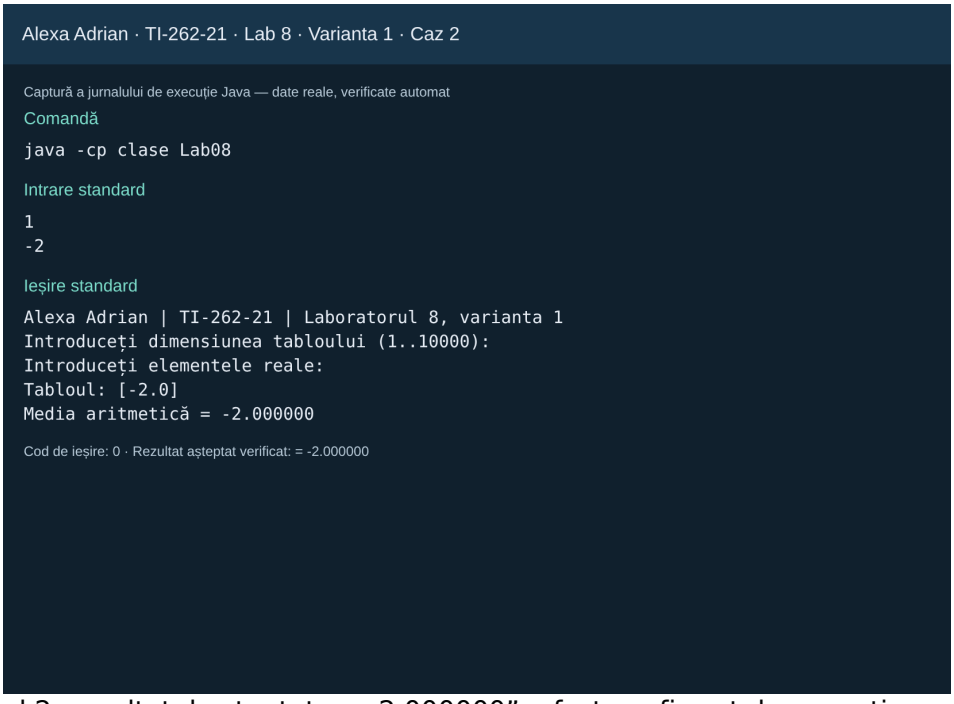


Figura 2. Cazul 2: rezultatul așteptat „= -2.000000” a fost confirmat de execuție.

Capturi de ecran și interpretare

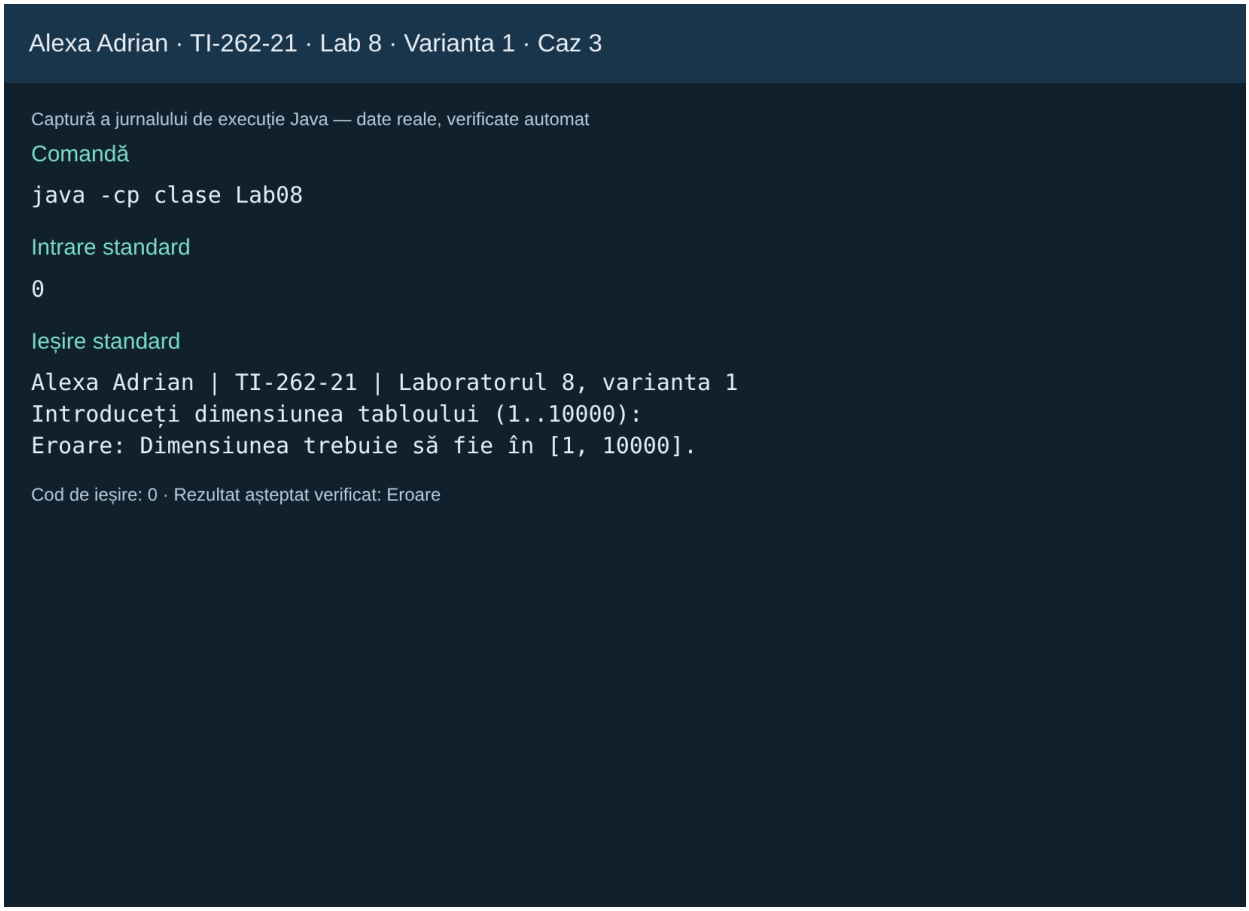


Figura 3. Cazul 3: rezultatul așteptat „Eroare” a fost confirmat de execuție.

Concluzii

1. Clasa Tablou reunește datele și operațiile într-un obiect.

2. Lab08 creează explicit obiectul și apelează metodele celeilalte clase.

3. Excepțiile opresc calculul pe date invalide și sunt tratate la nivelul interfeței.

Întrebări pentru pregătirea susținerii

• Care este diferența dintre clasă și obiect?

• De ce elemente este private?

• Pot exista două clase într-un fișier Java?

Reproducerea lucrării

Din rădăcina arhivei: javac -encoding UTF-8 -d clase cod/Lab08.java, apoi java -cp clase Lab08. Este necesar un JDK 21 sau mai nou. Pentru Lab09, un argument opțional stabilește calea fișierului TSV; fără argument se folosește studenti.tsv în directorul curent.

Surse

1. „Lucrările de laborator la disciplina Bazele Programării în Java”, anul 2024–2025, actualizare 16.09.2024, PDF furnizat pentru această lucrare.

2. Documentația Java SE 21: https://docs.oracle.com/en/java/javase/21/docs/api/ — clasele Scanner, Math, String, Files și BigInteger, după caz.

Varianta electronică include sursa integrală, capturile PNG, jurnalele TXT, fișierele de date și instrucțiunile de rulare. Datele de prezentare, nota și semnătura se completează la susținere. Emblema de pe copertă este o reproducere grafică după imaginea furnizată de student.

MINISTERUL EDUCAȚIEI ȘI CERCETĂRII AL REPUBLICII MOLDOVA

UNIVERSITATEA LIBERĂ INTERNAȚIONALĂ DIN MOLDOVA

ULIM



Facultatea Informatică, Inginerie și Design

Catedra Tehnologii Informaționale

RAPORT

Lucrarea de laborator nr. 9 · Varianta 1

Disciplina: Bazele Programării în Java

Tema: Fișiere

Anul de studii al cerințelor: 2024–2025

A elaborat: Alexa Adrian · Grupa TI-262-21

Profesor: Mititelu Vitalii

Data prezentării: _____

Nota: _____ Semnătura profesorului: _____

Chișinău

Scopul și sarcinile lucrării

Citirea datelor studenților de la tastatură, salvarea și recitirea lor dintr-un fișier și afișarea studenților cu media mai mare sau egală cu 8.

Material teoretic

Fișierele păstrează datele după închiderea programului. Pachetul java.nio.file oferă Path pentru identificarea căii și Files pentru citire și scriere. Codificarea UTF-8 este specificată explicit, pentru a păstra diacriticele independent de configurarea sistemului.

Înregistrarea unui student conține numărul, numele, prenumele, localitatea, adresa, telefonul și media. Un record Java definește compact acest tip de date și generează metodele de acces. Telefonul este text, pentru a nu pierde zerourile inițiale. Pentru teste se folosesc date demonstrative, nu date de contact reale.

Formatul ales este TSV: fiecare linie reprezintă un student, iar tabulatorul separă cele șapte câmpuri. Interfața interzice câmpurile goale și tabulatorii, pentru a nu rupe structura. Numărul trebuie să fie pozitiv și unic. Media este finită și aparține intervalului [1, 10]. La încărcare sunt verificate structura, numerele și duplicatele; un fișier deteriorat este raportat, nu interpretat ca fiind valid.

Meniul oferă creare, afișare, adăugare, modificare, eliminare, filtrare și ieșire. Crearea unui fișier gol cere confirmare, deoarece suprascrie conținutul. Adăugarea respinge un număr existent, modificarea cere un număr deja prezent, iar eliminarea raportează absența. Filtrarea recitește fișierul și selectează medie >= 8.0, astfel încât nota exact 8 trebuie inclusă. Pentru fiecare opțiune datele sunt reîncărcate de pe disc; operația de filtrare nu se bazează doar pe obiectele rămase în memorie.

Convenții de lucru: surse în UTF-8, Java 21, intrare de la tastatură și mesaje în limba română. Separatorul zecimal la tastare este punctul. Cazurile de test au fost executate efectiv; jurnalele brute și codul sunt incluse în varianta electronică.

Algoritmul și verificarea rezultatului

Fluxul este: alegere opțiune → încărcare fișier → validare și operație → salvare dacă datele s-au modificat. Opțiunea 6 parcurge înregistrările și afișează numai mediile ≥ 8.

Testul principal salvează două înregistrări cu mediile 8 și 7,99. La filtrare apare exact o înregistrare: cea cu media 8. Testul de validare introduce media 11 și confirmă că nu este salvată. Testul de modificare schimbă media 9 în 7, verifică filtrul, apoi elimină studentul. Un test suplimentar verifică numărul duplicat.

Lista din memorie ocupă O(N) spațiu. Filtrarea și salvarea sunt O(N); detectarea duplicatelor la încărcare este $O(N^2)$, acceptabilă pentru o grupă mică. Fișierul este text local, fără acces concurent sau salvare tranzacțională. Datele externe se verifică înainte de utilizare; o eroare de disc este raportată prin IOException.

Planul de testare și rezultatele

Cazul 1: 1 | DA | 3 | 1 | Alexa | Adrian | Chișinău | Strada Exemplu 1 | 000000000 | 8 | 3 | 2 | Test | Student | Chișinău | Strada Exemplu… (intrarea completă se află în jurnal). Verificare: „Înregistrări afișate: 1”. Rezultat: trecut; cod de ieșire 0.

Cazul 2: 1 | DA | 3 | 1 | Alexa | Adrian | Chișinău | Strada Exemplu 1 | 000000000 | 11 | 6 | 0. Verificare: „Înregistrări afișate: 0”. Rezultat: trecut; cod de ieșire 0.

Cazul 3: 1 | DA | 3 | 1 | Alexa | Adrian | Chișinău | Strada Exemplu 1 | 000000000 | 9 | 4 | 1 | Alexa | Adrian | Chișinău | Strada Exemplu… (intrarea completă se află în jurnal). Verificare: „Student eliminat.”. Rezultat: trecut; cod de ieșire 0.

Cazul 4: 1 | DA | 3 | 1 | Alexa | Adrian | Chișinău | Strada Exemplu 1 | 000000000 | 8 | 3 | 1 | Alexa | Adrian | Chișinău | Strada Exemplu… (intrarea completă se află în jurnal). Verificare: „Numărul există deja.”. Rezultat: trecut; cod de ieșire 0.

Pentru fiecare caz sunt păstrate separat intrarea, ieșirea și rezultatul așteptat. Capturile reproduc jurnalele reale într-o pagină de vizualizare; nu simulează un IDE. La lucrarea 9, capturile arată finalul jurnalului, iar fișierele TXT păstrează sesiunea integrală.

Codul integral — Lab09.java

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
import java.util.Locale;
import java.util.Scanner;

public class Lab09 {
    record Student(int nr, String nume, String prenume, String localitate,
                   String adresa, String telefon, double medie) {
        String linie() {
            return nr + "\t" + nume + "\t" + prenume + "\t" + localitate
                    + "\t" + adresa + "\t" + telefon + "\t" + medie;
        }
    }
    private static String camp(Scanner in, String mesaj) {
        System.out.println(mesaj);
        String s = in.nextLine().trim();
        if (s.isEmpty() || s.contains("\t")) {
            throw new IllegalArgumentException("Câmp gol sau tabulator nepermis.");
        }
        return s;
    }
    private static Student citeste(Scanner in) {
        int nr = Integer.parseInt(camp(in, "Numărul studentului:"));
        String nume = camp(in, "Numele:");
        String prenume = camp(in, "Prenumele:");
        String loc = camp(in, "Localitatea:");
        String adresa = camp(in, "Adresa:");
        String tel = camp(in, "Telefonul:");
        double medie = Double.parseDouble(camp(in, "Nota medie (1..10):"));
        if (nr < 1 || !Double.isFinite(medie) || medie < 1 || medie > 10) {
            throw new IllegalArgumentException("Număr pozitiv și medie în [1, 10]
    necesare.");
        }
        return new Student(nr, nume, prenume, loc, adresa, tel, medie);
    }
    private static List<Student> incarca(Path p) throws IOException {
        List<Student> lista = new ArrayList<>();
        if (!Files.exists(p)) return lista;
        for (String linie : Files.readAllLines(p, StandardCharsets.UTF_8)) {
            String[] c = linie.split("\t", -1);
            if (c.length != 7) throw new IOException("Fișier TSV deteriorat.");
```

Codul integral — Lab09.java (continuare)

```
            try {
                Student s = new Student(Integer.parseInt(c[0]), c[1], c[2],
                        c[3], c[4], c[5], Double.parseDouble(c[6]));
                if (s.nr() < 1 || !Double.isFinite(s.medie())
                        || s.medie() < 1 || s.medie() > 10) {
                    throw new IOException("Valori invalide în fișier.");
                }
                for (Student vechi : lista) {
                    if (vechi.nr() == s.nr()) throw new IOException("Număr duplicat în
    fișier.");
                }
                lista.add(s);
            } catch (NumberFormatException e) {
                throw new IOException("Număr invalid în fișier.", e);
            }
        }
        return lista;
    }
    private static void salveaza(Path p, List<Student> lista) throws IOException {
        List<String> linii = new ArrayList<>();
        for (Student s : lista) linii.add(s.linie());
        Files.write(p, linii, StandardCharsets.UTF_8);
    }
    private static void afiseaza(List<Student> lista, boolean filtrare) {
        int gasiti = 0;
        for (Student s : lista) {
            if (!filtrare || s.medie() >= 8.0) {
                System.out.println(s.linie());
                gasiti++;
            }
        }
        System.out.println("Înregistrări afișate: " + gasiti);
    }
    public static void main(String[] args) throws IOException {
        Locale.setDefault(Locale.ROOT);
        Scanner in = new Scanner(System.in);
        Path p = Path.of(args.length == 0 ? "studenti.tsv" : args[0]);
        System.out.println("Alexa Adrian | TI-262-21 | Laboratorul 9, varianta 1");
        System.out.println("Fișier UTF-8: " + p);
        while (true) {
            System.out.println("1 Creare | 2 Afișare | 3 Adăugare | 4 Modificare");
            System.out.println("5 Eliminare | 6 Studenți cu media >= 8 | 0 Ieșire");
            if (!in.hasNextLine()) return;
            String opt = in.nextLine().trim();
            if (opt.equals("0")) return;
```

Codul integral — Lab09.java (continuare)

```
            try {
                List<Student> lista = incarca(p);
                switch (opt) {
                    case "1" -> {
                        System.out.println("Recreare fișier: scrieți DA pentru
    confirmare.");
                        if (in.nextLine().equals("DA")) {
                            salveaza(p, new ArrayList<>());
                            System.out.println("Fișier creat.");
                        } else System.out.println("Creare anulată.");
                    }
                    case "2" -> afiseaza(lista, false);
                    case "3", "4" -> {
                        Student s = citeste(in);
                        int poz = -1;
                        for (int i = 0; i < lista.size(); i++) {
                            if (lista.get(i).nr() == s.nr()) poz = i;
                        }
                        if (opt.equals("3") && poz >= 0) {
                            throw new IllegalArgumentException("Numărul există deja.");
                        }
                        if (opt.equals("4") && poz < 0) {
                            throw new IllegalArgumentException("Studentul nu există.");
                        }
                        if (poz < 0) lista.add(s); else lista.set(poz, s);
                        salveaza(p, lista);
                        System.out.println("Date salvate.");
                    }
                    case "5" -> {
                        int nr = Integer.parseInt(camp(in, "Numărul de eliminat:"));
                        boolean eliminat = lista.removeIf(s -> s.nr() == nr);
                        if (eliminat) salveaza(p, lista);
                        System.out.println(eliminat ? "Student eliminat." : "Student
    inexistent.");
                    }
                    case "6" -> afiseaza(lista, true);
                    default -> System.out.println("Opțiune invalidă.");
                }
            } catch (IllegalArgumentException | IOException e) {
                System.out.println("Eroare: " + e.getMessage());
            }
        }
    }
}
```

Capturi de ecran și interpretare

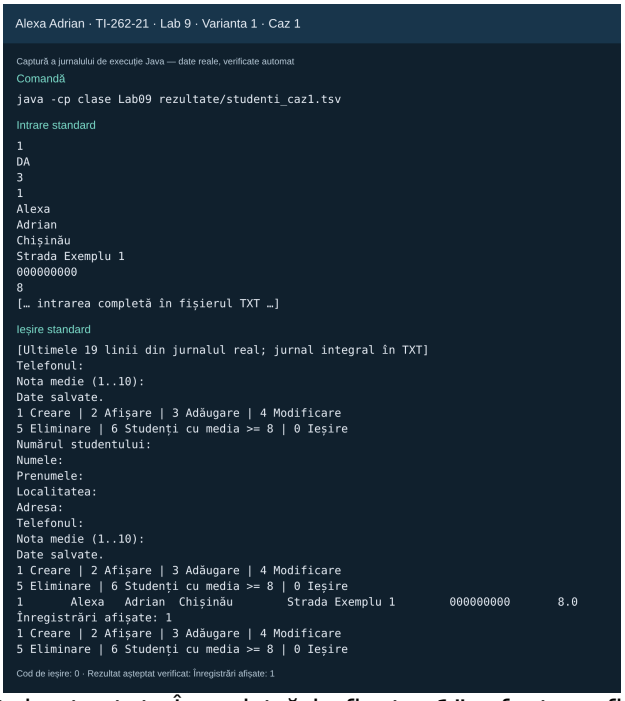


Figura 1. Cazul 1: rezultatul așteptat „Înregistrări afișate: 1” a fost confirmat de execuție.

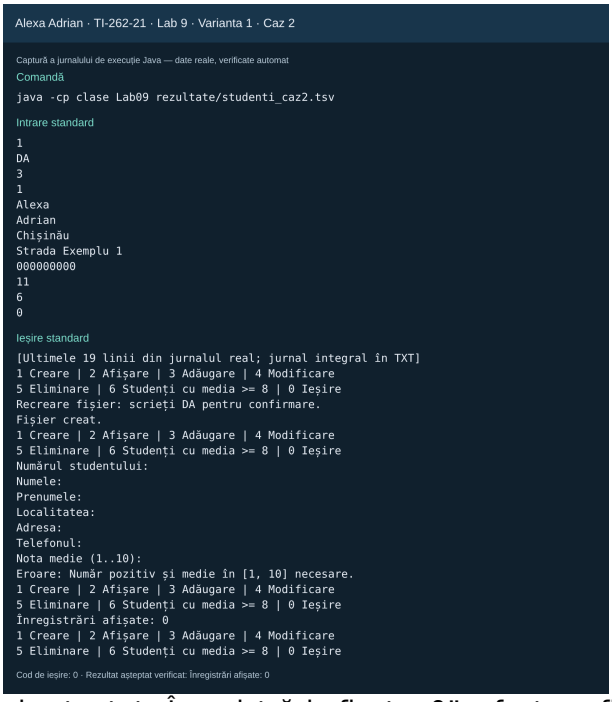


Figura 2. Cazul 2: rezultatul așteptat „Înregistrări afișate: 0” a fost confirmat de execuție.

Capturi de ecran și interpretare

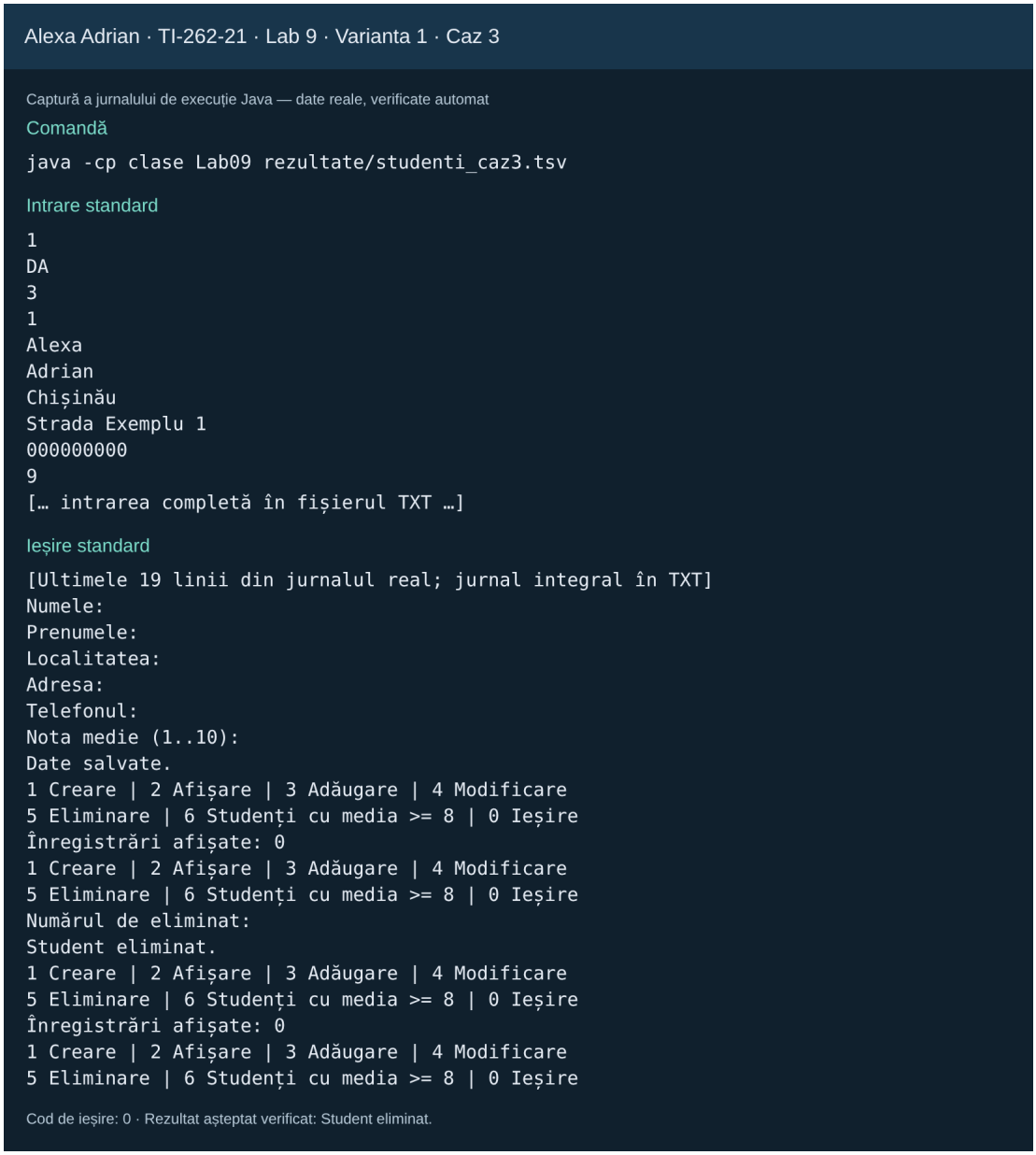


Figura 3. Cazul 3: rezultatul așteptat „Student eliminat.” a fost confirmat de execuție.

Concluzii

1. UTF-8 și formatul TSV păstrează structura și diacriticele în fișier.

2. Filtrul include media exact 8 și exclude valoarea 7,99, conform variantei 1.

3. Meniul permite modificarea și eliminarea datelor, iar validarea respinge mediile și numerele invalide.

Întrebări pentru pregătirea susținerii

• De ce telefonul este String?

• De ce filtrul folosește >= și nu >?

• Ce se întâmplă dacă un câmp conține tabulator?

Reproducerea lucrării

Din rădăcina arhivei: javac -encoding UTF-8 -d clase cod/Lab09.java, apoi java -cp clase Lab09. Este necesar un JDK 21 sau mai nou. Pentru Lab09, un argument opțional stabilește calea fișierului TSV; fără argument se folosește studenti.tsv în directorul curent.

Surse

1. „Lucrările de laborator la disciplina Bazele Programării în Java”, anul 2024–2025, actualizare 16.09.2024, PDF furnizat pentru această lucrare.

2. Documentația Java SE 21: https://docs.oracle.com/en/java/javase/21/docs/api/ — clasele Scanner, Math, String, Files și BigInteger, după caz.

Varianta electronică include sursa integrală, capturile PNG, jurnalele TXT, fișierele de date și instrucțiunile de rulare. Datele de prezentare, nota și semnătura se completează la susținere. Emblema de pe copertă este o reproducere grafică după imaginea furnizată de student.